|| Jai Sri Gurudev ||

Sri Adichunchanagiri Shikshana Trust (R)

SJB INSTITUTE OF TECHNOLOGY

SJBIT

Question Bank

All Modules

Subject Name: Python for Data Analytics

Subject Code: 23CSE312

Faculty

Dr. Veena H N, Associate Professor

Mrs. Shilpashree S, Assistant Professor

Semester: III

Department of Computer Science & Engineering

Aca. Year: Odd Sem /2025-26

Question Bank with Scheme and Solution

Sl. No.	Question - Scheme & Solution	Marks
	Module 1	
1.	**Define the data analysis process and explain its key stages with a neat block diagram.**	**10 Marks**
	Scheme: ● Definition of data analysis (2 marks) ● Listing and explanation of key stages (6 marks) ● Block diagram (2 marks) **Solution:** ● Data analysis is the transformation of raw data into useful information and models to understand systems and predict their behavior. ● Key stages: 1. Problem definition 2. Data extraction 3. Data cleaning 4. Data transformation 5. Data exploration 6. Predictive modeling 7. Model validation/test 8. Visualization and interpretation 9. Deployment ● **Diagram: Figure 1-1 in the textbook**	
2.	**What are the different types of predictive models in data analysis? Discuss their applications.**	**10 Marks**
	Scheme: ● Types of predictive models (4 marks) ● Application areas (6 marks) **Solution:** ● Types: 1. Classification models – output is categorical (e.g., spam detection). 2. Regression models – output is numeric (e.g., sales forecasting). 3. Clustering models – output is descriptive (e.g., customer segmentation). ● Applications span fields like business, science, and social systems for forecasting and decision-making.	
3.	**What are the main objectives of predictive modeling in data analysis.**	**10 Marks**
	Scheme: ● Purpose (2 marks) ● Main objectives (8 marks)	

	Solution: ● Objective: Build models to predict future outcomes based on data. ● Key objectives: ○ Forecasting system behavior ○ Supporting decision-making ○ Understanding relationships among variables ○ Classifying or grouping data ○ Estimating probabilities ○ Measuring risk and impact ○ Automating pattern recognition ○ Improving system insights	
4.	**Discuss the evolution of data analysis from simple data protection to a formal discipline. Why are models important in this context?**	**10 Marks**
	Scheme: ● Evolution overview (5 marks) ● Importance of models (5 marks) **Solution:** ● Started as basic data handling → evolved into a discipline with formal methodologies and modeling. ● Models convert systems into mathematical/logical forms enabling predictions and insights. ● Their predictive power is the ultimate goal, not the model itself.	
5.	**Why is Python considered a preferred language for data analysis?**	**10 Marks**
	Scheme: ● Features of Python (6 marks) ● Comparison and benefits (4 marks) **Solution:** ● Python offers a rich ecosystem of libraries (e.g., NumPy, Pandas), supports data processing, visualization, and modeling. ● It's easy to integrate with other languages, supports web development, and is widely adopted in scientific communities. ● Compared to R/Matlab, it's more versatile and future-ready.	
6.	**Explain the difference between quantitative and qualitative data analysis, providing examples of each.**	**10 Marks**
	Scheme: ● Quantitative analysis (4 marks) ● Qualitative analysis (4 marks) ● Examples (2 marks) **Solution:** ● **Quantitative**: Deals with numeric or categorical structured data. Supports objective, mathematical processing (e.g., survey scores).	

	● **Qualitative**: Deals with unstructured descriptive data, includes subjective interpretation (e.g., interview transcripts). ● Quantitative → predictions; Qualitative → understanding complex systems.	
7.	**Discuss about knowledge domains of the data analyst.**	**10 Marks**
	Scheme: ● Key domains (6 marks) ● Importance and application (4 marks) **Brief Solution:** ● A good data analyst needs knowledge in: 1. **Computer Science** – for tools, coding, data extraction (e.g., Python, SQL, Web Scraping) 2. **Mathematics & Statistics** – for modeling and analysis (e.g., regression, Bayesian methods) 3. **Machine Learning & AI** – for automated pattern detection 4. **Application domains** – for context understanding (e.g., biology, finance) ● Enables effective, interdisciplinary problem-solving.	
8.	**Describe the various types of data used in data analysis and explain their key characteristics.**	**10 Marks**
	Scheme: ● Categories of data (6 marks) ● Characteristics and examples (4 marks) **Solution:** ● **Categorical Data:** ○ *Nominal*: no order (e.g., gender) ○ *Ordinal*: ordered categories (e.g., rating scales) ● **Numerical Data:** ○ *Discrete*: countable (e.g., number of items) ○ *Continuous*: measurable within a range (e.g., height) ● **Characteristics: structure, measurability, and type define suitable analysis methods.**	
	Module 2	
1.	List the key features of Python that make it a popular programming language.	**5 Marks**
	Scheme: List and explain any 10 features – **0.5 mark each** (Total: 5 Marks) **Solution:** 1. Python has simple and easy-to-understand syntax; it is developer-friendly and high-level. 2. It supports branching, looping, and modular programming through functions. 3. Efficient in handling both numeric and textual data. 4. Provides dynamic and automatic memory management. 5. Offers a wide range of operators and functions for data structures like lists, tuples, and dictionaries.	

	6. Includes a comprehensive collection of tools for data manipulation, analysis, and processing. 7. Python code is interpreted, which aids in quick development and testing. 8. It is platform-independent and runs across various operating systems. 9. Python is dynamically typed; variable types are determined at runtime. 10. Comes with built-in graphical support and software integration capabilities. 11. Offers extensive standard and third-party libraries for specialized tasks. 12. Widely used in machine learning and data analytics due to its powerful ML libraries.	
2.	What are the naming rules for variables in programming? Illustrate how to assign values to variables. Explain with examples. (5 marks)	**5 Marks**
	Scheme: ● Naming rules (3) ● Assignment example (2) **Solution:** Rules: ● Names must start with a letter or underscore. ● Cannot use keywords. ● Can contain letters, digits, underscores. ● Cannot use punctuation characters such as @, # and % ● Examples. Assigning Values to variable: ● An identifier is associated with an expression or a value. ● The = operator means assignment and does not represent mathematical equality. ● Examples: name = "Alice", _age = 25	
3.	How do you take user input in Python using the input() function? Explain with examples how type conversion and multiple inputs can be handled.	**5 Marks**
	Scheme: ● input() syntax (1) ● Type conversion (2) ● Multiple inputs (2) **Solution:** **Examples:** name = input("Enter your name: ") age = int(input("Enter your age: ")) a, b = input("Enter two numbers: ").split() a, b = int(a), int(b)	
4.	How is output handled in Python using the print() function? Illustrate the following with examples : a. end b. sep c. Formatting options f-strings and str.format().	**5 Marks**

	Scheme:
	● Basic print (1)
	● end & sep (1)
	● f-strings (1.5)
	● str.format() (1.5)
	Solution:
	print("Hello", end="!") # end example
	print("A", "B", sep="-") # sep example
	name = "John"
	print(f'Hello, {name}") # f-string
	print("Hello, {}".format(name)) # str.format()	

5.	Explain the different types of operators in Python with examples.	**5 Marks**
	Scheme: ● Arithmetic (1) ● Relational (1) ● Logical (1) ● Assignment (1) ● Membership/Identity (1) **Solution:** # Arithmetic x + y, x - y # Relational x > y, x == y # Logical x and y, not x # Assignment x += 5 # Membership/Identity 3 in [1, 2, 3], x is y	
6.	Explain the use of if, if-else, and if-elif-else statements with flowchart and examples.	**5 Marks**

	Scheme:
	● Syntax of each (3)
	● Example + flowchart mention (2)
	Solution:
	if x > 0:
	print("Positive")
	elif x == 0:
	print("Zero")
	else:
	print("Negative")
	Flowchart: Decision tree with branches for each condition	

7.	Explain the use of for and while loops with flowchart and examples.	**7 Marks**

	Scheme:
	● Syntax of for (1)
	● Syntax of while (1)
	● Examples (2)
	● Flowchart mention (1)
	Solution:
	for i in range(3):
	print(i)
	count = 0
	while count < 3:
	print(count)
	count += 1
	Flowchart: Loop with condition check and iteration	

8.	Explain abnormal loop termination in Python. Provide examples to demonstrate their usage.	**5 Marks**

	Scheme:
	● break (2)
	● continue (2)
	● pass (1)
	Solution:
	for i in range(5):
	if i == 3:
	break
	print(i)
	
	for i in range(5):
	if i == 2:
	continue
	print(i)
	
	for i in range(5):
	pass # Placeholder	

9.	Explain key characteristics of lists in Python. Demonstrate any 7 built-in list functions with code examples for each.	**10 Marks**
	Scheme: ● Characteristics (3) ● 7 functions with examples (7) **Solution:** **Lists are ordered, mutable, and allow duplicate elements.** l = [1, 2, 3] l.append(4)	

	l.insert(1, 5) l.remove(2) l.pop() l.sort() l.reverse() len(l)	
10.	What is a list in Python? Demonstrate how to access elements of a list using indexing and slicing with examples.	**5 Marks**
	Scheme: ● Definition (1) ● Indexing (2) ● Slicing (2) **Solution:** lst = [10, 20, 30, 40] print(lst[1]) # Indexing print(lst[-1]) # Last element print(lst[1:3]) # Slicing	
11.	Describe the purpose of the + and * operators in Python lists, and provide examples of their usage.	**5 Marks**
	Scheme: ● + for concatenation (2 marks) ● * for repetition (2 marks) ● Example (1 mark) **Solution:** lst1 = [1, 2] lst2 = [3, 4] print(lst1 + lst2) print(lst1 * 2)	
12.	What are tuples in Python, and how are they created? Explain with examples.	**5 Marks**
	Scheme: ● Definition (1 mark) ● Creation (2 marks) ● Examples (2 marks) **Solution:** Tuples are ordered, immutable collections. t1 = (1, 2, 3) t2 = tuple([4, 5]) t3 = ("a",)	
13.	How can elements in a tuple be accessed? Demonstrate accessing tuple elements using indexing	**5 Marks**

	and slicing.	

	Scheme:
	● Indexing (2 marks)
	● Slicing (2 marks)
	● Example (1 mark)
	Solution:
	t = (10, 20, 30, 40)
	print(t[0]) # Indexing
	print(t[-1])
	print(t[1:3]) # Slicing	

14.	What are the key differences between tuples and lists in Python?	**5 Marks**
	Scheme: ● Mutability (2 marks) ● Syntax (1 mark) ● Use case & performance (2 marks) **Solution:** ● Lists are mutable; tuples are immutable. ● Lists use [], tuples use (). ● Tuples are faster and used for fixed data.	
15.	Explain the different built-in functions available for tuples in Python, providing examples for each.	**6 Marks**
	Scheme: ● len() (1 mark) ● max() and min() (2 marks) ● sum() (1 mark) ● count() and index() (2 marks) **Solution:** t = (1, 2, 3, 2) print(len(t)) print(max(t), min(t)) print(sum(t)) print(t.count(2)) print(t.index(3))	
16.	What is a dictionary in Python? Describe its key characteristics and use cases. Demonstrate five different ways to create a dictionary, providing code examples for each method.	**10 Marks**
	Scheme: ● Definition and characteristics (3 marks) ● 5 creation methods with code (7 marks)	

	Solution:Dictionary is a key-value mapping structure. Keys are unique and immutable. d1 = {"a": 1, "b": 2} d2 = dict([("a", 1), ("b", 2)]) d3 = dict(a=1, b=2) d4 = dict(zip(["a", "b"], [1,2])) d5 = {} ; d5["a"] = 1; d5["b"] = 2	
17.	What are the key features of dictionaries in Python, and how do the keys(), values(), and items() methods work? Provide examples to illustrate their usage.	**6 Marks**
	Scheme: ● Key features (2 marks) ● keys(), values(), items() usage (4 marks) **Solution:** Dictionaries are unordered, mutable, key-value stores. d = {"x": 10, "y": 20} print(d.keys()) print(d.values()) print(d.items())	
18.	Describe the various built-in functions available for dictionaries in Python, along with examples for each	**6 Marks**
	Scheme: ● get(), update() (2 marks) ● pop(), clear() (2 marks) ● in, del (2 marks) **Solution:** d = {"a": 1, "b": 2} print(d.get("a")) d.update({"c": 3}) d.pop("b") d.clear() "a" in d # del d["a"]	
	Module 3	
1.	**Explain the significance of ndarray in NumPy. Write code for creating 1D and 2D Arrays and demonstrate the following operations:** ● **Identify the data type of the elements in your array.** ● **Find the number of dimensions in the created array.** ● **Calculate the total number of elements in your array.** ● **Determine the shape of the array.** ● **Find the size of each element in bytes.** ● **Identify the axes of the array.** ● **Tabulate the data types supported by ndarray in NumPy along with their descriptions.**	**10 Marks**

	Scheme: ● Significance of ndarray (3 marks): Discuss its advantages over Python lists (homogeneity, memory efficiency, speed, mathematical operations). ● Array Creation (2 marks): Code for 1D and 2D arrays. ● Demonstrate Operations (3 marks): Code and output for dtype, ndim, size, shape, itemsize, axes (conceptual understanding of axes). ● Data Types Tabulation (2 marks): Table with common NumPy data types and descriptions. **Solution:** The ndarray (N-dimensional array) is the core data structure in NumPy. Its significance lies in providing: ● Homogeneity: All elements in an ndarray must be of the same data type, enabling highly optimized operations. ● Memory Efficiency: ndarray stores elements in contiguous memory blocks, reducing memory overhead compared to Python lists. ● Performance: Optimized C implementations for array operations lead to significantly faster computations than Python loops. ● Mathematical Operations: Enables efficient element-wise operations and matrix manipulations crucial for numerical computing.	
2.	**Demonstrate the following with code examples:** ● **Create a two-dimensional array filled with zeros.** ● **Create a two-dimensional array filled with ones.** ● **Generate a sequence of values between 0 and 10 (arange()).** ● **Generate a two-dimensional array using arange() combined with reshape().** ● **Generate a sequence of numbers using linspace() that divides the interval between 0 and 10 into five equal parts.**	**7 Marks**

	Scheme:
	● Each bullet point: 1.4 marks (code and output).
	Solution:
	import numpy as np
	
	# Create a two-dimensional array filled with zeros (3x4)
	zeros_array = np.zeros((3, 4))
	print("2D Array of Zeros:\n", zeros_array)
	
	# Create a two-dimensional array filled with ones (2x3)
	ones_array = np.ones((2, 3))
	print("\n2D Array of Ones:\n", ones_array)
	
	# Generate a sequence of values between 0 and 10 (exclusive of 10)
	arange_sequence = np.arange(0, 10)
	print("\nSequence using arange(0, 10):", arange_sequence)
	
	# Generate a two-dimensional array using arange() combined with reshape()	

	# Create 12 elements and reshape into a 3x4 array
	reshaped_array = np.arange(12).reshape((3, 4))
	print("\nReshaped array (3x4) from arange:\n", reshaped_array)
	
	# Generate a sequence of numbers using linspace() that divides the interval between 0 and 10 into five equal parts
	linspace_sequence = np.linspace(0, 10, 5)
	print("\nSequence using linspace(0, 10, 5):", linspace_sequence)	

3.	**Discuss the basic arithmetic operations that can be performed on NumPy arrays. Illustrate your explanation with code examples demonstrating the following:** ● **Perform scalar addition and multiplication on a one-dimensional array.** ● **Demonstrate element-wise operations between two arrays.** ● **Explain the difference between element-wise operations and matrix multiplication, providing code examples for both using the dot() function.** ● **Show how to use increment and decrement operators on a NumPy array.**	**10 Marks**

	Scheme:
	● **Discussion of basic arithmetic operations (2 marks):** Mention element-wise nature.
	● **Scalar operations (2 marks):** Code for addition and multiplication.
	● **Element-wise operations between arrays (2 marks):** Code for various operations.
	● **Element-wise vs. Matrix Multiplication (3 marks):** Explanation and code for * and dot().
	● **Increment/Decrement (1 mark):** Code examples.
	Solution:
	NumPy arrays allow for basic arithmetic operations (addition, subtraction, multiplication, division) to be performed efficiently. These operations are inherently **element-wise**, meaning the operation is applied independently to each corresponding element of the arrays involved.
	import numpy as np
	
	# Sample 1D array
	arr = np.array([1, 2, 3, 4, 5])
	print("Original 1D Array:", arr)
	
	# Perform scalar addition
	arr_plus_5 = arr + 5
	print("Array + 5 (scalar addition):", arr_plus_5)
	
	# Perform scalar multiplication
	arr_times_2 = arr * 2
	print("Array * 2 (scalar multiplication):", arr_times_2)
	
	# Demonstrate element-wise operations between two arrays
	arr1 = np.array([10, 20, 30])
	arr2 = np.array([1, 2, 3])
	print("\nArray 1:", arr1)	

```
print("Array 2:", arr2)

print("Element-wise Addition (arr1 + arr2):", arr1 + arr2)
print("Element-wise Subtraction (arr1 - arr2):", arr1 - arr2)
print("Element-wise Multiplication (arr1 * arr2):", arr1 * arr2)
print("Element-wise Division (arr1 / arr2):", arr1 / arr2)

# Explain the difference between element-wise operations and matrix multiplication
print("\n--- Element-wise vs. Matrix Multiplication ---")

# Element-wise multiplication: Multiplies corresponding elements.
# Requires arrays to have compatible shapes (broadcasting rules apply).
# Uses the '*' operator.
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])
print("Matrix 1:\n", matrix1)
print("Matrix 2:\n", matrix2)

print("\nElement-wise multiplication (matrix1 * matrix2):\n", matrix1 * matrix2)

# Matrix multiplication: Follows linear algebra rules for matrix product.
# For matrices A (m x n) and B (n x p), the result is C (m x p).
# The number of columns in the first matrix must equal the number of rows in the second
matrix.
# Uses the np.dot() function or the '@' operator (Python 3.5+).
print("\nMatrix multiplication (np.dot(matrix1, matrix2)):\n", np.dot(matrix1, matrix2))

# Show how to use increment and decrement operators on a NumPy array
array_for_ops = np.array([10, 20, 30])
print("\nOriginal array for increment/decrement:", array_for_ops)

array_for_ops += 1 # Equivalent to array_for_ops = array_for_ops + 1
print("After increment by 1:", array_for_ops)

array_for_ops -= 5 # Equivalent to array_for_ops = array_for_ops - 5
print("After decrement by 5:", array_for_ops)
```

4.	**Describe universal functions (ufuncs) and provide examples with at least three different mathematical operations.**	**5 Marks**
	Scheme: ● **Definition of ufuncs (2 marks):** Explain their purpose (element-wise operations, speed, type promotion). ● **Examples (3 marks):** Code for at least three different mathematical ufuncs. **Solution:**	

	Universal functions (ufuncs) in NumPy are functions that operate element-wise on ndarrays. They are "universal" because they are designed to work across various data types and dimensions, applying the same operation to each element independently. Ufuncs are implemented in highly optimized C code, making them significantly faster than standard Python loops for numerical computations. They also handle broadcasting and type promotion automatically.
	import numpy as np
	
	# Sample array
	arr_ufunc = np.array([1.5, 2.7, 3.2, 4.9, 5.1])
	print("Original array for ufuncs:", arr_ufunc)
	
	# Example 1: np.sqrt() - Calculate the square root of each element
	sqrt_arr = np.sqrt(arr_ufunc)
	print("Square root (np.sqrt()):", sqrt_arr)
	
	# Example 2: np.exp() - Calculate the exponential of each element (e^x)
	exp_arr = np.exp(arr_ufunc)
	print("Exponential (np.exp()):", exp_arr)
	
	# Example 3: np.log() - Calculate the natural logarithm of each element
	log_arr = np.log(arr_ufunc)
	print("Natural Logarithm (np.log()):", log_arr)
	
	# Example 4: np.sin() - Calculate the sine of each element
	sin_arr = np.sin(arr_ufunc)
	print("Sine (np.sin()):", sin_arr)	

5.	**Explain aggregate functions in NumPy and demonstrate the use of at least three aggregate functions on a sample array.**	**5 Marks**
	Scheme: ● **Explanation of aggregate functions (2 marks):** Discuss their purpose (summarizing data, reducing dimensions). ● **Demonstration (3 marks):** Code for at least three aggregate functions (sum, mean, max, min, std, etc.). **Solution:** Aggregate functions (or reduction operations) in NumPy compute a single value that summarizes an array or a specific axis of an array. They are essential for data analysis as they allow you to quickly derive insights like totals, averages, maximums, minimums, and standard deviations from large datasets. Unlike ufuncs, which operate element-wise, aggregate functions reduce the number of dimensions of the array. import numpy as np # Sample array	

	data_array = np.array([[10, 20, 30],
	[40, 50, 60],
	[70, 80, 90]])
	print("Original Array:\n", data_array)
	
	# Example 1: np.sum() - Calculate the sum of all elements
	total_sum = np.sum(data_array)
	print("\nSum of all elements:", total_sum)
	
	# Example 2: np.mean() - Calculate the mean (average) of all elements
	average_value = np.mean(data_array)
	print("Mean of all elements:", average_value)
	
	# Example 3: np.max() - Find the maximum value in the array
	max_value = np.max(data_array)
	print("Maximum value in array:", max_value)
	
	# Example 4: np.min() - Find the minimum value in the array
	min_value = np.min(data_array)
	print("Minimum value in array:", min_value)
	
	# Example 5: np.std() - Calculate the standard deviation
	std_dev = np.std(data_array)
	print("Standard deviation of all elements:", std_dev)
	
	# Aggregate functions can also operate along specific axes
	print("\nSum along axis 0 (columns sum for each row):", np.sum(data_array, axis=0))
	print("Mean along axis 1 (rows mean for each column):", np.mean(data_array, axis=1))	

6.	**Consider the following NumPy array and perform the specified tasks:** import numpy as np A = np.arange(10, 19).reshape((3, 3)) ● **Access the element located in the second row and the third column of the array A.** ● **Slice the array A to obtain all elements in the first row.** ● **Extract the first two rows and the first two columns of the array A.** ● **Iterate over the elements of array A and print each element.**	**7 Marks**
	Scheme: Each bullet point: 1.75 marks (code and output). **Solution**: import numpy as np A = np.arange(10, 19).reshape((3, 3)) print("Original Array A:\n", A) # Access the element located in the second row and the third column (0-indexed)	

```
# Second row is index 1, third column is index 2
element = A[1, 2]
print("\nElement at second row, third column:", element)

# Slice the array A to obtain all elements in the first row
# First row is index 0. ':' indicates all columns.
first_row = A[0, :]
print("Elements in the first row:", first_row)

# Extract the first two rows and the first two columns of the array A
# First two rows: 0:2 (exclusive of 2)
# First two columns: 0:2 (exclusive of 2)
sub_array = A[0:2, 0:2]
print("First two rows and first two columns:\n", sub_array)

# Iterate over the elements of array A and print each element
print("\nIterating over elements of A:")
for element_val in A.flatten(): # .flatten() creates a 1D copy for easier iteration
    print(element_val, end=" ")
print() # for newline

# Alternative iteration (row by row then element by element)
print("\nIterating row by row, then element by element:")
for row in A:
    for element_val in row:
        print(element_val, end=" ")
    print() # new line for each row
```

7.	**Explain how slicing differs in NumPy compared to standard Python lists and describe the functionality of the apply_along_axis() method.**	**7 Marks**

Scheme:

- **Slicing difference (4 marks):** Discuss views vs. copies, multi-dimensional slicing.
- **apply_along_axis() (3 marks):** Explanation of functionality and purpose.

Solution:

Slicing in NumPy vs. Standard Python Lists:

The primary difference in slicing between NumPy arrays and standard Python lists is how they handle memory:

1. **Views vs. Copies:**
    - **NumPy Arrays:** Slicing a NumPy array generally returns a **view** of the original array. This means that no new memory is allocated for the slice; it's simply a new way of looking at the same underlying data. Modifications to the slice will directly affect the original array. This behavior is crucial for memory efficiency when working with large datasets.

	○ **Python Lists:** Slicing a Python list creates a **new list**, which is a **copy** of the original slice. Any modifications to the sliced list will *not* affect the original list. 2. **Multi-dimensional Slicing:** ○ **NumPy Arrays:** NumPy arrays support multi-dimensional slicing using comma-separated indices, making it intuitive to access rows, columns, or sub-arrays (e.g., arr[row_slice, col_slice]). ○ **Python Lists:** Python lists are inherently one-dimensional. To simulate multi-dimensional slicing (e.g., list of lists), you'd typically need nested slicing (e.g., list_of_lists[row_index][col_index]), which can be less efficient and verbose for complex selections. **Functionality of apply_along_axis():** The numpy.apply_along_axis(func1d, axis, arr, *args, **kwargs) method is a powerful function that applies a 1-D function (func1d) to slices of an array (arr) along a specified axis. ● **func1d**: This is the function that will be applied. It must be able to operate on a 1-dimensional array. ● **axis**: This specifies the axis along which func1d will be applied. For example, axis=0 applies the function column-wise, and axis=1 applies it row-wise (for a 2D array). ● **arr**: The input array. ● ***args, **kwargs**: Optional arguments to pass to func1d. **Purpose:** apply_along_axis() is useful when you have a custom function that operates on 1D data, and you want to apply that function across rows, columns, or other higher-dimensional "slices" of a multi-dimensional array, without explicitly writing loops. It effectively automates the looping process for you. While often convenient, it's generally slower than vectorized NumPy operations (ufuncs or methods like sum(), mean()) if a vectorized alternative exists.	
8.	**Create a 4x4 NumPy array containing random numbers between 0 and 1.** ● **Create a Boolean array that identifies all elements in the array A that are less than 0.5.** ● **Extract and display all elements from A that are less than 0.5.**	**5 Marks**

	Scheme:
	● **Array creation (1 mark):** Code for random 4x4 array.
	● **Boolean array (2 marks):** Code and output.
	● **Extraction (2 marks):** Code and output.
	Solution:
	import numpy as np
	
	# Create a 4x4 NumPy array containing random numbers between 0 and 1
	A = np.random.rand(4, 4)
	print("Original 4x4 Random Array A:\n", A)
	
	# Create a Boolean array that identifies all elements in the array A that are less than 0.5
	boolean_array = A < 0.5
	print("\nBoolean array (elements < 0.5):\n", boolean_array)
	
	# Extract and display all elements from A that are less than 0.5	

	extracted_elements = A[boolean_array] # Using boolean indexing
	print("\nElements from A less than 0.5:", extracted_elements)	

9.	**Explain the difference between horizontal stacking and vertical stacking in NumPy.** ● **Explain how the stacking and splitting functions in NumPy facilitate flexible manipulation of data structures. Provide suitable code examples to illustrate your explanation.**	**10 Marks**

	Scheme:
	● **Horizontal vs. Vertical Stacking (2 marks):** Clear explanation.
	● **Role of Stacking/Splitting in Data Manipulation (3 marks):** Explanation with general examples.
	● **vstack() and hstack() code (2 marks):** Correct application and output.
	● **Horizontal split (1.5 marks):** Code and output.
	● **Vertical split (1.5 marks):** Code and output.
	Solution:
	Difference between Horizontal Stacking and Vertical Stacking:
	● **Vertical Stacking (vstack)**: Concatenates arrays along the **vertical axis (axis 0)**, effectively adding rows. For vstack to work, the arrays must have the same number of columns.
	● **Horizontal Stacking (hstack)**: Concatenates arrays along the **horizontal axis (axis 1)**, effectively adding columns. For hstack to work, the arrays must have the same number of rows.
	How Stacking and Splitting Functions Facilitate Flexible Manipulation:
	NumPy's stacking (vstack, hstack, dstack, concatenate) and splitting (vsplit, hsplit, dsplit, array_split) functions are crucial for flexible manipulation of data structures because they allow you to:
	● **Combine data from different sources:** You might have data stored in separate arrays (e.g., features from different sensors, or different time series) that you want to analyze together. Stacking allows you to merge these into a single, larger array.
	● **Prepare data for specific operations:** Some algorithms or functions expect data in a particular shape (e.g., a single large matrix). Stacking can help achieve this.
	● **Divide large datasets:** For processing efficiency or to distribute work, you might need to split a large array into smaller, manageable chunks. Splitting functions enable this.
	● **Reorganize data:** You can reshape and then stack/split arrays to change their overall structure, which is often necessary in data preprocessing pipelines.
	Example
	import numpy as np
	
	# Create two 3x3 arrays
	A = np.ones((3, 3))
	B = np.zeros((3, 3))
	print("Array A (ones):\n", A)
	print("\nArray B (zeros):\n", B)	

	# Use vstack() to vertically stack A and B
	stacked_v = np.vstack((A, B))
	print("\nVertically stacked (vstack) A and B:\n", stacked_v)
	
	# Use hstack() to horizontally stack A and B
	stacked_h = np.hstack((A, B))
	print("\nHorizontally stacked (hstack) A and B:\n", stacked_h)
	
	# Create a 4x4 matrix CCC
	CCC = np.arange(16).reshape((4, 4))
	print("\nOriginal 4x4 matrix CCC:\n", CCC)
	
	# Split the matrix CCC horizontally into two parts
	# Requires an even division, so 2 parts is fine for a 4-column matrix
	split_h1, split_h2 = np.hsplit(CCC, 2)
	print("\nHorizontal split part 1:\n", split_h1)
	print("\nHorizontal split part 2:\n", split_h2)
	
	# Split the matrix CCC into three parts along the vertical axis at indices [1, 3]
	# This splits before index 1, then before index 3.
	# The resulting parts are columns 0; columns 1-2; columns 3.
	split_v1, split_v2, split_v3 = np.vsplit(CCC, [1, 3])
	print("\nVertical split part 1 (columns up to index 0):\n", split_v1)
	print("\nVertical split part 2 (columns from index 1 to 2):\n", split_v2)
	print("\nVertical split part 3 (columns from index 3 onwards):\n", split_v3)	

10.	**Discuss the importance of reading from and writing to files in NumPy for data analysis.** ● **Write a code sample to demonstrate the following:** ○ **Create an array and save it in a binary file format.** ○ **Load the data from the binary file and display the loaded array.** ○ **Read data from a CSV file containing tabular data into a structured array.** ○ **Display the structured array and extract the values of a specific column using its header.**	**10 Marks**
	Scheme: ● **Importance (3 marks):** Discuss persistence, large datasets, data exchange. ● **Binary save/load (3 marks):** Code and output. ● **CSV read (structured array) (4 marks):** Code for CSV creation, reading, structured array display, column extraction. **Solution:** **Importance of Reading from and Writing to Files in NumPy for Data Analysis:** Reading from and writing to files is paramount in data analysis using NumPy for several reasons: 1. **Data Persistence:** Once data is generated or processed, it often needs to be saved for future use, analysis, or sharing. Files provide a persistent storage mechanism, preventing	

data loss when the program terminates.

2. **Handling Large Datasets:** Datasets in real-world scenarios are often too large to fit entirely into memory. Reading data in chunks or writing processed results incrementally to files is essential for memory management.
3. **Data Exchange:** Files serve as a common medium for exchanging data between different applications, programming languages, or collaborators who might not be using NumPy directly (e.g., CSV, text files).
4. **Reproducibility:** Saving intermediate and final results allows for reproducibility of analyses, enabling others (or your future self) to verify results or build upon previous work without re-running entire pipelines.
5. **Offline Processing:** Data can be collected offline, stored in files, and then loaded into NumPy for analysis at a later time.

```
import numpy as np
import os # For file path operations

# --- Demonstrate saving to and loading from binary file (.npy) ---

# Create an array
my_array = np.arange(10, 20).reshape((2, 5))
print("Original array for binary save/load:\n", my_array)

# Define filename
binary_filename = "my_numpy_array.npy"

# Save the array in a binary file format (.npy is NumPy's native format)
np.save(binary_filename, my_array)
print(f"\nArray saved to {binary_filename}")

# Load the data from the binary file
loaded_array = np.load(binary_filename)
print(f"Array loaded from {binary_filename}:\n", loaded_array)

# Clean up the created file
# os.remove(binary_filename)
# print(f"Cleaned up {binary_filename}")

# --- Demonstrate reading data from a CSV file into a structured array ---

# 1. Create a dummy CSV file for demonstration
csv_filename = "data.csv"
csv_content = """Name,Age,Score
Alice,25,88.5
Bob,30,92.0
Charlie,22,79.3
"""
with open(csv_filename, 'w') as f:
```

	f.write(csv_content)
	print(f"\nCreated dummy CSV file: {csv_filename}")
	
	# 2. Read data from the CSV file into a structured array
	# use np.genfromtxt for more control, especially with mixed data types
	# For structured arrays, define dtype or let it infer (less robust for mixed types)
	data = np.genfromtxt(csv_filename, delimiter=',', dtype=None, names=True, encoding=None)
	# names=True reads the header and uses them as field names
	# dtype=None attempts to infer data types
	# encoding=None to avoid decoding issues on some systems
	
	print(f"\nStructured array loaded from {csv_filename}:\n", data)
	
	# Display the structured array (it often looks like a table)
	print("\nData type of the structured array:", data.dtype)
	
	# Extract the values of a specific column using its header
	ages = data['Age']
	print("\n'Age' column values:", ages)
	
	# Clean up the created CSV file
	# os.remove(csv_filename)
	# print(f"Cleaned up {csv_filename}")	

11.	**Explain why ndarray is essential in NumPy. What advantages does it bring to data processing and analysis?**	**7 Marks**
	Scheme: ● **Essentiality (2 marks):** Core data structure. ● **Advantages (5 marks):** Discuss at least 5 key advantages (efficiency, speed, functionality, homogeneity, memory layout). **Solution:** The ndarray is the fundamental and essential object in NumPy because it forms the backbone for all numerical operations and data representation within the library. It's the primary reason NumPy is so powerful for scientific computing and data analysis in Python. Advantages it brings to data processing and analysis: 1. **Memory Efficiency:** ndarray stores elements in contiguous blocks of memory. Unlike Python lists, which store pointers to objects scattered in memory, ndarray stores actual data values contiguously. This leads to significantly less memory overhead, especially for large datasets. 2. **Speed (Vectorization):** NumPy operations on ndarrays are implemented in highly optimized C and Fortran code, avoiding Python's slow loops. This "vectorization" means operations are performed on entire arrays at once, providing orders of magnitude speed improvement over pure Python code. 3. **Homogeneity:** All elements in an ndarray must be of the same data type. This strict type enforcement allows NumPy to perform very efficient, type-specific operations and	

	enables memory optimization. 4. **Rich Functionality:** ndarray is designed to work seamlessly with a vast collection of functions and methods provided by NumPy for linear algebra, Fourier transforms, random number generation, and more. This extensive toolkit is built around the ndarray. 5. **Broadcasting:** ndarray supports broadcasting, a powerful mechanism that allows NumPy to work with arrays of different shapes when performing arithmetic operations, making code more concise and efficient without explicit looping or tiling. 6. **Multi-Dimensionality:** ndarray supports N-dimensional arrays (vectors, matrices, and higher-order tensors), which are crucial for representing and manipulating various types of scientific and engineering data.	
12.	**List and explain five essential properties of NumPy arrays (e.g., shape, dtype, dimensions, size, and itemsize) and how they help in data analysis.**	**5 Marks**
	Scheme: ● Each property: 1 mark (name, explanation, and relevance to data analysis). **Solution:** Five essential properties of NumPy arrays: 1. **shape**: ○ **Explanation:** A tuple that indicates the size of the array in each dimension. For example, a 2x3 matrix will have a shape of (2, 3). ○ **Relevance in Data Analysis:** Understanding the shape is crucial for ensuring compatibility in operations (e.g., matrix multiplication, concatenation), for reshaping data into required formats (e.g., for machine learning models), and for quickly grasping the organization of the dataset. 2. **dtype**: ○ **Explanation:** The data type of the elements in the array. Since ndarrays are homogeneous, all elements share the same type (e.g., int32, float64, bool). ○ **Relevance in Data Analysis:** dtype is vital for memory management (e.g., using float32 instead of float64 for less precision to save memory). It also impacts precision in calculations and helps identify potential data conversion issues or unexpected data types. 3. **ndim (Number of Dimensions)**: ○ **Explanation:** The number of dimensions (axes) of the array. A scalar has ndim=0, a vector has ndim=1, a matrix has ndim=2, and so on. ○ **Relevance in Data Analysis:** Knowing the number of dimensions helps in conceptualizing the data structure. It guides how you perform operations along specific axes and how you might need to reshape the array to match the input requirements of functions or models (e.g., a neural network might expect a 2D input where each row is a sample). 4. **size**: ○ **Explanation:** The total number of elements in the array. It's the product of the elements of the shape tuple. ○ **Relevance in Data Analysis:** size gives you an immediate count of all data points, which is important for estimating memory consumption, understanding the scale of the dataset, and for progress tracking in iterative algorithms.	

	5. **itemsize**: ○ **Explanation:** The size in bytes of each element of the array. For example, an array of float64 will have itemsize=8 bytes. ○ **Relevance in Data Analysis:** itemsize directly relates to memory usage. By multiplying size by itemsize, you can calculate the total memory consumed by the array. This is critical for optimizing memory for very large datasets and for understanding the underlying data representation.	
13.	**What are some typical ways to create arrays in NumPy? Provide an example of creating arrays using zeros(), ones(), arange(), and linspace().**	**7 Marks**

	Scheme:
	● **Typical ways (2 marks):** List common methods (from Python lists, zeros, ones, arange, linspace, empty, full, random).
	● **Examples (5 marks):** 1.25 marks for each code example.
	Solution:
	Typical ways to create arrays in NumPy include:
	● **From Python lists or tuples:** Using np.array().
	● **Creating arrays filled with specific values:** np.zeros(), np.ones(), np.full().
	● **Creating arrays with sequences of numbers:** np.arange(), np.linspace().
	● **Creating empty or uninitialized arrays:** np.empty().
	● **Creating arrays with random numbers:** Functions from np.random module (e.g., np.random.rand(), np.random.randint()).
	import numpy as np
	
	# Creating an array using np.zeros()
	zeros_array = np.zeros((2, 3)) # Creates a 2x3 array of zeros
	print("Array created with np.zeros():\n", zeros_array)
	
	# Creating an array using np.ones()
	ones_array = np.ones((3, 2)) # Creates a 3x2 array of ones
	print("\nArray created with np.ones():\n", ones_array)
	
	# Creating an array using np.arange()
	# Generates values from 0 up to (but not including) 10, with a step of 2
	arange_array = np.arange(0, 10, 2)
	print("\nArray created with np.arange(0, 10, 2):", arange_array)
	
	# Creating an array using np.linspace()
	# Generates 5 evenly spaced values between 0 and 1 (inclusive)
	linspace_array = np.linspace(0, 1, 5)
	print("\nArray created with np.linspace(0, 1, 5):", linspace_array)	

14.	**Describe the process of reshaping arrays in NumPy. Why is reshaping beneficial? Provide a sample code that reshapes a one-dimensional array into a 2x3 matrix.**	**6 Marks**

Scheme:

- **Process of reshaping (2 marks):** Explain reshape() method, total elements consistency.
- **Why beneficial (2 marks):** Discuss adapting data to algorithms, visualization, memory optimization.
- **Code example (2 marks):** Correct reshape operation.

Solution:

Process of Reshaping Arrays in NumPy:

Reshaping an array in NumPy involves changing its shape (dimensions) without changing the data or the total number of elements. The reshape() method is used for this purpose. When you reshape an array, NumPy simply interprets the existing data in the array with a new arrangement of rows and columns. Crucially, the total number of elements in the original array must be equal to the product of the dimensions in the new shape. If one of the dimensions in the new shape is -1, NumPy automatically calculates that dimension based on the array's size and the other specified dimensions.

Why is Reshaping Beneficial?

Reshaping is highly beneficial in data analysis and machine learning for several reasons:

1. **Adapting Data to Algorithm Requirements:** Many machine learning models and statistical algorithms expect input data in a specific shape (e.g., a 2D array where rows are samples and columns are features). Reshaping allows you to format your data correctly.
2. **Data Preprocessing and Transformation:** It's a fundamental step in data preprocessing, enabling you to transform data from one layout to another that is more suitable for analysis or visualization.
3. **Memory Efficiency:** Reshaping does not create a copy of the data (unless explicitly requested or if it's not possible to create a view), making it memory-efficient for large datasets. It merely changes the way NumPy "views" the existing data.
4. **Broadcast Compatibility:** Reshaping can sometimes be used to make arrays compatible for broadcasting operations.

Example:import numpy as np

```
# Create a one-dimensional array with 6 elements
original_1d_array = np.arange(6)
print("Original 1D array:", original_1d_array)
print("Original shape:", original_1d_array.shape)

# Reshape the 1D array into a 2x3 matrix
# Total elements: 6 (2 * 3 = 6)
reshaped_2x3_matrix = original_1d_array.reshape((2, 3))
print("\nReshaped 2x3 matrix:\n", reshaped_2x3_matrix)
print("New shape:", reshaped_2x3_matrix.shape)

# Another example: using -1 to infer a dimension
reshaped_inferred = original_1d_array.reshape((3, -1)) # -1 infers the second dimension as 2
print("\nReshaped matrix (3x-1):\n", reshaped_inferred)
print("New shape (inferred):", reshaped_inferred.shape)
```

15.	**How does NumPy handle element-wise operations compared to scalar operations? Provide examples of both.**	**6 Marks**

Scheme:

- **Element-wise operations (2 marks):** Explanation.
- **Scalar operations (2 marks):** Explanation.
- **Examples (2 marks):** Code for both.

Solution:

NumPy handles element-wise and scalar operations very efficiently, leveraging its C-implemented core.

Element-wise Operations: When performing operations between two NumPy arrays of compatible shapes, the operation is applied **element by element** to corresponding positions. This means the first element of array A is operated with the first element of array B, the second with the second, and so on. The result is a new array with the same shape as the input arrays (or a broadcasted shape). This is a core strength of NumPy, allowing for very fast computations without explicit Python loops.

Scalar Operations: When a NumPy array is operated with a scalar (a single number), the scalar value is effectively "broadcast" across the entire array. This means the scalar is applied to **every single element** of the array individually. NumPy intelligently handles this without actually creating a new array of scalars; it's an optimized internal process.

Examples:

```
import numpy as np

# --- Scalar Operations ---
scalar = 10
arr_scalar = np.array([1, 2, 3, 4, 5])
print("Original Array for Scalar Ops:", arr_scalar)
print("Scalar value:", scalar)

# Scalar addition
result_scalar_add = arr_scalar + scalar
print("Array + Scalar (addition):", result_scalar_add)

# Scalar multiplication
result_scalar_mult = arr_scalar * scalar
print("Array * Scalar (multiplication):", result_scalar_mult)

# --- Element-wise Operations ---
arr_elem1 = np.array([10, 20, 30, 40, 50])
arr_elem2 = np.array([1, 2, 3, 4, 5])
print("\nArray 1 for Element-wise Ops:", arr_elem1)
print("Array 2 for Element-wise Ops:", arr_elem2)

# Element-wise addition
result_elem_add = arr_elem1 + arr_elem2
print("Array 1 + Array 2 (element-wise addition):", result_elem_add)
```

	# Element-wise multiplication
	result_elem_mult = arr_elem1 * arr_elem2
	print("Array 1 * Array 2 (element-wise multiplication):", result_elem_mult)
	
	# Element-wise comparison
	result_elem_comp = arr_elem1 > arr_elem2
	print("Array 1 > Array 2 (element-wise comparison):", result_elem_comp)	

16.	**How are aggregate functions like sum, mean, and max useful in data analysis? Provide examples of each with a brief explanation.**	**7 Marks**
	Scheme: ● **Usefulness (2 marks):** Discuss summarization, quick insights, reducing data. ● **Examples (5 marks):** 1.66 marks for each function (explanation, code, output). **Solution:** Aggregate functions (also known as reduction functions) like sum, mean, and max are incredibly useful in data analysis because they allow you to quickly summarize large datasets, derive key statistics, and reduce data dimensionality, providing quick insights. They transform an array (or a slice of an array) into a single, representative value. Here are examples of their usefulness: **np.sum() (Summation):** ● **Explanation:** Calculates the sum of all elements in an array or along a specified axis. Useful for finding total counts, total values, or overall magnitudes. ● **Usefulness:** For example, calculating the total sales from a list of daily sales figures, or the total number of votes received across different polling stations. **np.mean() (Mean/Average):** ● **Explanation:** Calculates the arithmetic mean (average) of the elements in an array. Provides a central tendency measure. ● **Usefulness:** Used to understand typical values, like average exam scores, average temperature over a period, or the average income in a demographic group. **np.max() (Maximum Value):** ● **Explanation:** Finds the largest element in an array. ● **Usefulness:** Identifying peak values, like the highest temperature recorded, the maximum stock price reached, or the strongest signal strength. np.min() works similarly for minimum values. Code Examples	
17.	**Explain the following functions: vstack(), hstack(), vsplit() and hsplit().**	**7 Marks**
	Scheme: ● Each function: 1.75 marks (explanation, purpose, and key requirement). **Solution:** These NumPy functions are used for concatenating (stacking) and dividing (splitting) arrays, providing flexible ways to manipulate data structures. 1. **np.vstack(tup) (Vertical Stack):**	

	○ **Explanation:** Stacks arrays in sequence vertically (row-wise). It takes a tuple or list of arrays as input. ○ **Purpose:** Combines arrays by adding them as new rows. This is useful when you have datasets with the same number of features (columns) but different observations (rows) that you want to merge. ○ **Key Requirement:** All input arrays must have the same number of dimensions and the same shape along all but the first axis (i.e., same number of columns for 2D arrays). 2. **np.hstack(tup) (Horizontal Stack):** ○ **Explanation:** Stacks arrays in sequence horizontally (column-wise). It also takes a tuple or list of arrays as input. ○ **Purpose:** Combines arrays by adding them as new columns. This is useful when you have different features (columns) for the same observations (rows) that you want to combine into a single dataset. ○ **Key Requirement:** All input arrays must have the same number of dimensions and the same shape along all but the last axis (i.e., same number of rows for 2D arrays). 3. **np.vsplit(ary, indices_or_sections) (Vertical Split):** ○ **Explanation:** Splits an array into multiple sub-arrays vertically (along axis 0). ○ **Purpose:** Divides an array into chunks along its rows. Useful for distributing data for parallel processing or for analyzing specific subsets of rows. ○ **Key Requirement:** ■ If indices_or_sections is an integer N, the array will be divided into N equal parts. This requires N to exactly divide the number of rows. ■ If indices_or_sections is a 1-D array of sorted integers, these indicate the row indices at which the array should be split. 4. **np.hsplit(ary, indices_or_sections) (Horizontal Split):** ○ **Explanation:** Splits an array into multiple sub-arrays horizontally (along axis 1). ○ **Purpose:** Divides an array into chunks along its columns. Useful for separating different features or for breaking down a wide dataset into more manageable parts. ○ **Key Requirement:** ■ If indices_or_sections is an integer N, the array will be divided into N equal parts. This requires N to exactly divide the number of columns. ■ If indices_or_sections is a 1-D array of sorted integers, these indicate the column indices at which the array should be split.	
18.	**What is the difference between array(), zeros(), and ones() functions in Numpy?**	**6 Marks**
	Scheme: ● Each function: 2 marks (primary purpose, input, and key characteristic). **Solution:** The array(), zeros(), and ones() functions in NumPy are all used for creating arrays, but they serve distinct purposes and take different arguments: 1. **np.array(object, dtype=None, ...):** ○ **Primary Purpose:** To create an array from an existing Python object, such as a	

	list, tuple, or another array-like structure. It's the most general-purpose array creation function when you have data already. ○ **Input:** Takes a single object (e.g., [1, 2, 3], ((1,2),(3,4))) whose elements will populate the array. ○ **Key Characteristic:** The values of the array are determined by the input object. NumPy infers the dtype if not explicitly provided. 2. **np.zeros(shape, dtype=float, ...):** ○ **Primary Purpose:** To create a new array of a given shape, filled entirely with zeros. ○ **Input:** Takes a shape (an integer or a tuple of integers) that defines the dimensions of the array. Optionally, dtype can be specified (defaults to float). ○ **Key Characteristic:** All elements are initialized to 0 (or 0.0 for float types). It's useful for pre-allocating memory for arrays that will be filled later, or for creating masks. 3. **np.ones(shape, dtype=float, ...):** ○ **Primary Purpose:** To create a new array of a given shape, filled entirely with ones. ○ **Input:** Takes a shape (an integer or a tuple of integers) that defines the dimensions of the array. Optionally, dtype can be specified (defaults to float). ○ **Key Characteristic:** All elements are initialized to 1 (or 1.0 for float types). Useful for similar purposes as zeros(), such as initializing weights in a neural network or creating arrays for scaling operations.	
19.	**What is broadcasting in Numpy, and how does it work with arrays of different shapes?**	**7 Marks**
	Scheme: ● **Definition of Broadcasting (3 marks):** Explain its concept, purpose (element-wise with different shapes), and efficiency. ● **How it works (4 marks):** Explain broadcasting rules (dimension compatibility, extending dimensions, dimension 1 compatibility). Provide a small conceptual example. **Solution:** **Broadcasting in NumPy:** Broadcasting is a powerful mechanism in NumPy that allows arithmetic operations to be performed on arrays of different shapes. It provides a way to perform element-wise operations efficiently without explicitly making copies of arrays to match their dimensions. Instead, NumPy "broadcasts" the smaller array across the larger array so that they have compatible shapes for the operation. This avoids unnecessary memory allocation and improves performance. **How it Works with Arrays of Different Shapes:** Broadcasting follows a set of strict rules to determine if two arrays are compatible for an operation: 1. **Dimension Compatibility (Right-to-Left):** NumPy compares the dimensions of the two arrays starting from the **trailing (rightmost) dimension** and moving left. 2. **Rule 1: Equal Dimensions:** If the dimensions are equal, they are compatible. 3. **Rule 2: One of Them is 1:** If one of the dimensions is 1, the dimension with size 1 is stretched or "broadcast" to match the other dimension's size. 4. **Rule 3: Missing Dimension:** If one array has fewer dimensions than the other, its shape	

	is prepended with ones on the left until the number of dimensions matches. Then, Rule 1 and Rule 2 are applied. 5. **Incompatibility:** If none of these rules apply (i.e., dimensions are neither equal nor one of them is 1), the arrays are considered incompatible, and a ValueError (broadcast error) is raised.	
20.	**How can you create an array with evenly spaced values using arange() and linspace()?**	**6 Marks**
	Scheme: ● **arange() explanation (3 marks):** Purpose, arguments, example. ● **linspace() explanation (3 marks):** Purpose, arguments, example. **Solution:** NumPy provides two primary functions for generating arrays with evenly spaced values: arange() and linspace(). They differ in how you specify the spacing. 1. **np.arange(start, stop, step, dtype=None):** ○ **Purpose:** Creates an array with evenly spaced values within a given interval. It is similar to Python's built-in range() function but returns a NumPy ndarray. ○ **Arguments:** ■ start: (Optional) The starting value of the sequence (inclusive). Defaults to 0. ■ stop: The end value of the sequence (exclusive). The generated values will not reach this number. ■ step: (Optional) The spacing between consecutive values. Defaults to 1. ■ dtype: (Optional) The data type of the output array. ○ **Key Characteristic:** You specify the *step size*. The number of elements generated depends on the start, stop, and step values. It's best used when you know the desired interval and the precise step between values. **2. np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None):** ● **Purpose:** Creates an array with a specified num of evenly spaced values over a given interval. ● **Arguments:** ○ start: The starting value of the sequence (inclusive). ○ stop: The end value of the sequence. ○ num: (Optional) The number of samples to generate. Defaults to 50. ○ endpoint: (Optional) If True (default), the stop value is included in the sequence. If False, it is excluded. ○ retstep: (Optional) If True, returns (samples, step) where step is the spacing between samples. ○ dtype: (Optional) The data type of the output array. ● **Key Characteristic:** You specify the *number of samples* you want. NumPy then calculates the appropriate step size to ensure those num samples are evenly distributed between start and stop. It's commonly used when you need a specific number of points within an interval for plotting, function evaluation, or simulations.	

	Module 4	
1.	**Discuss the key characteristics and operations associated with Series and DataFrame objects in pandas.**	**12 Marks**
	Scheme: • **Series (6 marks):** Definition, key characteristics (1D, indexed, homogeneous), and common operations (creation, indexing, arithmetic). • **DataFrame (6 marks):** Definition, key characteristics (2D, labeled rows/columns, heterogeneous columns), and common operations (creation, indexing, column/row manipulation, arithmetic). **Solution:** Pandas introduces two fundamental data structures for data manipulation and analysis: Series and DataFrame. **Series Object:** A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It's essentially a column in a spreadsheet or a single column from a database table. **Key Characteristics:** • **One-dimensional:** It holds data in a single sequence. • **Labeled Index:** Each element in a Series has an associated label, called an index, which can be used for accessing data. If no index is provided, a default integer index (0 to N-1) is created. • **Homogeneous Data (typically):** While a Series *can* hold mixed data types (resulting in dtype=object), it's generally most efficient and useful when it contains data of a single, consistent type. • **Size Immutable, Value Mutable:** Once created, you cannot change the number of elements in a Series, but you can change the values of its elements. **Common Operations:** • **Creation:** From lists, arrays, dictionaries, or scalar values. • **Indexing and Slicing:** Accessing elements by their labels or by integer position, similar to NumPy arrays. • **Arithmetic Operations:** Element-wise operations with scalars or other Series (with alignment by index). • **Mathematical Functions:** Application of NumPy ufuncs or Series methods like sum(), mean(). • **Filtering:** Using boolean indexing. **DataFrame Object:** A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It's similar to a spreadsheet, a SQL table, or a dictionary of Series objects. It is the most commonly used pandas object. **Key Characteristics:** • **Two-dimensional:** It has both rows and columns. • **Labeled Axes:** It has both a row index (similar to Series) and a column index (column names). • **Heterogeneous Columns:** Unlike NumPy arrays, different columns in a DataFrame can	

	hold different data types (e.g., one column of integers, another of strings, another of booleans). However, elements *within* a single column are typically of the same type. ● **Size Mutable:** You can add or remove columns and rows. **Common Operations:** ● **Creation:** From dictionaries of lists/Series, 2D NumPy arrays, other DataFrames, CSV/Excel files, etc. ● **Column Selection:** Accessing columns using dictionary-like notation (df['column_name']). ● **Row Selection:** Using .loc (label-based) or .iloc (integer-position based) for row access. ● **Indexing and Slicing:** Powerful multi-axis indexing for selecting subsets of data. ● **Arithmetic Operations:** Element-wise operations between DataFrames, DataFrames and Series (with broadcasting and alignment). ● **Data Alignment:** Automatically aligns data based on row and column labels during operations, handling missing labels gracefully. ● **Missing Data Handling:** Built-in methods for identifying, filling, or dropping missing values (NaN). ● **Reshaping and Pivoting:** Functions to change the layout of the DataFrame (e.g., pivot_table, melt). ● **Aggregation:** Grouping data and applying aggregate functions (groupby()). ● **Joining/Merging:** Combining DataFrames based on common columns or indices.	
2.	**How are arithmetic operations performed between two DataFrames? What happens when the indexes and column names don't match?**	**10 Marks**
	Scheme: ● **How arithmetic operations are performed (4 marks):** Explain element-wise nature and automatic alignment. ● **Handling non-matching indexes/columns (6 marks):** Detail NaN generation and fill_value parameter. **Solution:** **How Arithmetic Operations are Performed Between Two DataFrames:** Arithmetic operations (addition, subtraction, multiplication, division, etc.) between two DataFrames in pandas are performed **element-wise**. This means that the operation is applied independently to each corresponding element in both DataFrames. The key feature of DataFrame arithmetic is **automatic data alignment**. Pandas automatically aligns the DataFrames based on both their **row indexes** and **column names**. Before performing the operation, pandas internally checks if the row labels and column labels of the two DataFrames match. **What Happens When the Indexes and Column Names Don't Match?** When the indexes or column names of the two DataFrames do not perfectly match, pandas handles this by introducing **missing values (NaN)**: 1. **Non-Matching Indexes/Columns:** ○ For any row index or column name that exists in one DataFrame but not in the other, pandas will include that index/column in the result. ○ For elements corresponding to these non-matching labels, the value will be filled with NaN (Not a Number) in the resulting DataFrame. This NaN signifies that	

	there was no corresponding value to perform the operation with. 2. **Example:** If df1 has a row with index 'A' and df2 does not, any arithmetic operation df1 + df2 will result in NaN for the elements in row 'A' in the output DataFrame where no matching column exists in df2. 3. **fill_value Parameter:** Pandas provides a fill_value parameter in its arithmetic methods (.add(), .sub(), .mul(), .div()) to explicitly control how non-matching values are handled. Instead of NaN, you can specify a numerical value to be used for missing entries. This value will be substituted in place of the NaN before the operation is performed. ○ df1.add(df2, fill_value=0): If df1 has an element at a label where df2 does not, that non-existent element in df2 will be treated as 0 for the addition. **Code Example**	
3.	**Explain the core indexing operations in pandas: reindexing, dropping, and alignment.**	**8 Marks**
	Scheme: ● **Reindexing (3 marks):** Explanation of reindex(), purpose (conform to new index), fill_value and method. ● **Dropping (3 marks):** Explanation of drop(), purpose (remove labels), axis, inplace. ● **Alignment (2 marks):** Recap automatic alignment during operations. **Solution:** Pandas provides powerful and flexible indexing operations crucial for data manipulation. Three core concepts are reindexing, dropping, and alignment. **1. Reindexing (.reindex()):** ● **Explanation:** Reindexing in pandas is the process of conforming a Series or DataFrame to a new index. It essentially rearranges the existing data to match a new set of labels, potentially introducing missing values (NaN) for labels not found in the original object or filling in existing data for labels that are present. ● **Purpose:** ○ Changing the order of rows or columns. ○ Adding or removing rows/columns. ○ Aligning data from different sources (e.g., combining time series data with different frequencies). ○ Filling in missing data using various methods. ● **Key Parameters:** ○ index (or columns for DataFrame): The new index/column labels to conform to. ○ fill_value: A scalar value to use for labels that were not present in the original object. ○ method: A filling method for missing data, such as 'ffill' (forward fill), 'bfill' (backward fill), or 'pad'. **Example** 2. **Dropping (.drop()):** ● **Explanation:** Dropping in pandas refers to the operation of removing rows or columns from a DataFrame or Series based on their labels. It returns a *new* object with the specified labels removed, by default.	

- **Purpose:** To clean or subset data by removing irrelevant rows (e.g., duplicate entries, rows with too many missing values) or columns (e.g., redundant features, columns with no predictive power).
- **Key Parameters:**
  - labels: The index labels (for rows) or column names (for columns) to drop. Can be a single label or a list of labels.
  - axis: Specifies whether to drop rows (axis=0 or 'index') or columns (axis=1 or 'columns').
  - inplace: (Optional, default False) If True, the operation modifies the DataFrame in place and returns None. If False, it returns a new DataFrame.

Example

Since the current date is July 7, 2025, and the user has provided a new set of questions, I will address these questions.

Here's the scheme and solution for the next set of Pandas questions, following the format and assumed mark distribution (5-10 marks, adjusted for complexity) based on "Data Analytics using Python - Bharto Motwani, Wiley publication."

Pandas Questions - Scheme and Solution

General Instructions:

- Solutions are concise, focusing on core concepts and code examples.
- Assumed mark distribution: 5-10 marks per question based on complexity and number of sub-parts.
- Refer to "Data Analytics using Python - Bharto Motwani, Wiley publication" for theoretical background and further details.

Question 1: Discuss the key characteristics and operations associated with Series and DataFrame objects in pandas.
(Marks: 12)

Scheme:

- **Series (6 marks):** Definition, key characteristics (1D, indexed, homogeneous), and common operations (creation, indexing, arithmetic).
- **DataFrame (6 marks):** Definition, key characteristics (2D, labeled rows/columns, heterogeneous columns), and common operations (creation, indexing, column/row manipulation, arithmetic).

Solution:

Pandas introduces two fundamental data structures for data manipulation and analysis: Series and DataFrame.

Series Object:

A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects, etc.). It's essentially a column in a spreadsheet or a single column from a database table.

Key Characteristics:

- **One-dimensional:** It holds data in a single sequence.

- **Labeled Index:** Each element in a Series has an associated label, called an index, which can be used for accessing data. If no index is provided, a default integer index (0 to N-1) is created.
- **Homogeneous Data (typically):** While a Series *can* hold mixed data types (resulting in dtype=object), it's generally most efficient and useful when it contains data of a single, consistent type.
- **Size Immutable, Value Mutable:** Once created, you cannot change the number of elements in a Series, but you can change the values of its elements.

Common Operations:

- **Creation:** From lists, arrays, dictionaries, or scalar values.
- **Indexing and Slicing:** Accessing elements by their labels or by integer position, similar to NumPy arrays.
- **Arithmetic Operations:** Element-wise operations with scalars or other Series (with alignment by index).
- **Mathematical Functions:** Application of NumPy ufuncs or Series methods like sum(), mean().
- **Filtering:** Using boolean indexing.

DataFrame Object:

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It's similar to a spreadsheet, a SQL table, or a dictionary of Series objects. It is the most commonly used pandas object.

Key Characteristics:

- **Two-dimensional:** It has both rows and columns.
- **Labeled Axes:** It has both a row index (similar to Series) and a column index (column names).
- **Heterogeneous Columns:** Unlike NumPy arrays, different columns in a DataFrame can hold different data types (e.g., one column of integers, another of strings, another of booleans). However, elements *within* a single column are typically of the same type.
- **Size Mutable:** You can add or remove columns and rows.

Common Operations:

- **Creation:** From dictionaries of lists/Series, 2D NumPy arrays, other DataFrames, CSV/Excel files, etc.
- **Column Selection:** Accessing columns using dictionary-like notation (df['column_name']).
- **Row Selection:** Using .loc (label-based) or .iloc (integer-position based) for row access.
- **Indexing and Slicing:** Powerful multi-axis indexing for selecting subsets of data.
- **Arithmetic Operations:** Element-wise operations between DataFrames, DataFrames and Series (with broadcasting and alignment).
- **Data Alignment:** Automatically aligns data based on row and column labels during operations, handling missing labels gracefully.
- **Missing Data Handling:** Built-in methods for identifying, filling, or dropping missing values (NaN).
- **Reshaping and Pivoting:** Functions to change the layout of the DataFrame (e.g., pivot_table, melt).
- **Aggregation:** Grouping data and applying aggregate functions (groupby()).
- **Joining/Merging:** Combining DataFrames based on common columns or indices.

Question 2: How are arithmetic operations performed between two DataFrames? What happens when the indexes and column names don't match?
(Marks: 10)
Scheme:

- **How arithmetic operations are performed (4 marks):** Explain element-wise nature and automatic alignment.
- **Handling non-matching indexes/columns (6 marks):** Detail NaN generation and fill_value parameter.

Solution:
How Arithmetic Operations are Performed Between Two DataFrames:
Arithmetic operations (addition, subtraction, multiplication, division, etc.) between two DataFrames in pandas are performed **element-wise**. This means that the operation is applied independently to each corresponding element in both DataFrames.
The key feature of DataFrame arithmetic is **automatic data alignment**. Pandas automatically aligns the DataFrames based on both their **row indexes** and **column names**. Before performing the operation, pandas internally checks if the row labels and column labels of the two DataFrames match.
What Happens When the Indexes and Column Names Don't Match?
When the indexes or column names of the two DataFrames do not perfectly match, pandas handles this by introducing **missing values (NaN)**:

1. **Non-Matching Indexes/Columns:**
    - For any row index or column name that exists in one DataFrame but not in the other, pandas will include that index/column in the result.
    - For elements corresponding to these non-matching labels, the value will be filled with NaN (Not a Number) in the resulting DataFrame. This NaN signifies that there was no corresponding value to perform the operation with.
2. **Example:** If df1 has a row with index 'A' and df2 does not, any arithmetic operation df1 + df2 will result in NaN for the elements in row 'A' in the output DataFrame where no matching column exists in df2.
3. **fill_value Parameter:** Pandas provides a fill_value parameter in its arithmetic methods (.add(), .sub(), .mul(), .div()) to explicitly control how non-matching values are handled. Instead of NaN, you can specify a numerical value to be used for missing entries. This value will be substituted in place of the NaN before the operation is performed.
    - df1.add(df2, fill_value=0): If df1 has an element at a label where df2 does not, that non-existent element in df2 will be treated as 0 for the addition.

Code Example:

```
import pandas as pd
import numpy as np

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]}, index=['x', 'y'])
df2 = pd.DataFrame({'B': [5, 6], 'C': [7, 8]}, index=['y', 'z'])

print("DataFrame 1:\n", df1)
print("\nDataFrame 2:\n", df2)
```

```
# Element-wise addition
# 'A' column from df1 has no match in df2 -> NaN
# 'C' column from df2 has no match in df1 -> NaN
# 'x' index from df1 has no match in df2 -> NaN
# 'z' index from df2 has no match in df1 -> NaN
df_sum = df1 + df2
print("\nResult of df1 + df2 (with NaNs for non-matches):\n", df_sum)
# Expected output:
#     A    B    C
# x  NaN  NaN  NaN
# y  NaN  9.0  NaN
# z  NaN  NaN  NaN

# Using fill_value
# Non-matching values will be treated as 0 for addition
df_sum_fill = df1.add(df2, fill_value=0)
print("\nResult of df1.add(df2, fill_value=0):\n", df_sum_fill)
# Expected output:
#     A    B    C
# x  1.0  3.0  7.0
# y  2.0  9.0  8.0
# z  NaN  6.0  8.0
# Note: 'A' in 'z' row, and 'C' in 'x' row are still NaN because the original DF didn't have those
indices/columns.
# The fill_value applies to the *missing corresponding element* in the other DataFrame.
# E.g., for df1.add(df2, fill_value=0):
# df1 is {A: {x:1, y:2}, B: {x:3, y:4}}
# df2 is {B: {y:5, z:6}, C: {y:7, z:8}}
# Result for index 'x', column 'A': df1['A']['x'] + (df2['A']['x'] which is missing, filled with 0) = 1
+ 0 = 1.0
# Result for index 'x', column 'B': df1['B']['x'] + (df2['B']['x'] which is missing, filled with 0) = 3
+ 0 = 3.0
# Result for index 'x', column 'C': (df1['C']['x'] which is missing, filled with 0) + (df2['C']['x']
which is missing, filled with 0) = 0 + 0 = 0.0 (error in my manual trace, should be 0.0 if both are
missing from both)
# The output shows NaN for 'A' and 'C' at index 'x', meaning `fill_value` only applies to
elements from `df2` that are missing in `df1` at a given intersection.
# Let's re-verify the behavior of fill_value. It imputes missing values *before* the operation.
# Correct interpretation: df1.add(df2, fill_value=X) treats NaNs in df1 as X if df2 has a value,
and NaNs in df2 as X if df1 has a value.
# If an index/column pair is completely absent from one DataFrame, it effectively means NaN
from that DataFrame.
# So, df1.add(df2, fill_value=0):
# For (x, A): df1 has 1, df2 has no A. df1.loc['x','A'] = 1, df2.loc['x','A'] = NaN (treated as 0).
Result: 1+0=1
```

```
# For (x, B): df1 has 3, df2 has no B at x. df1.loc['x','B'] = 3, df2.loc['x','B'] = NaN (treated as 0). Result: 3+0=3
# For (x, C): df1 has no C at x, df2 has no C at x. This means df1.loc['x','C'] = NaN, df2.loc['x','C'] = NaN.
# If both are missing, `fill_value` does *not* create a value for the sum of two implicit zeros. It remains NaN.
# The `fill_value` argument fills `NaN` values *in the DataFrame being operated on* before the operation.
# Re-testing:
df_test1 = pd.DataFrame({'A': [1, np.nan], 'B': [3, 4]})
df_test2 = pd.DataFrame({'A': [5, 6], 'B': [np.nan, 8]})
print("\nDataFrame test1:\n", df_test1)
print("\nDataFrame test2:\n", df_test2)
print("\nResult of df_test1.add(df_test2, fill_value=0):\n", df_test1.add(df_test2, fill_value=0))
# This correctly gives [6, 6] for A and [3, 12] for B.
# My prior conceptualization for df1/df2 was wrong regarding `fill_value` and entirely new labels.
# For df1 + df2: pandas creates the union of index and column labels.
# For each (index, col) pair in the union:
#   If present in df1, use its value. Else, use NaN.
#   If present in df2, use its value. Else, use NaN.
# Then, the operation (e.g., addition) is performed. If either operand is NaN, the result is NaN.
# When using `df1.add(df2, fill_value=X)`:
#   For an (index, col) present in df1 but not in df2, df2's value is effectively X.
#   For an (index, col) present in df2 but not in df1, df1's value is effectively X.
#   If it's present in both or neither, the fill_value doesn't apply (for the *intermediate* values, it does for the final NaN rule).
# So my df_sum_fill example was correct in its output.
```

Question 3: Explain the core indexing operations in pandas: reindexing, dropping, and alignment.
(Marks: 8)
Scheme:

- **Reindexing (3 marks):** Explanation of reindex(), purpose (conform to new index), fill_value and method.
- **Dropping (3 marks):** Explanation of drop(), purpose (remove labels), axis, inplace.
- **Alignment (2 marks):** Recap automatic alignment during operations.

Solution:
Pandas provides powerful and flexible indexing operations crucial for data manipulation. Three core concepts are reindexing, dropping, and alignment.
1. Reindexing (.reindex()):

- **Explanation:** Reindexing in pandas is the process of conforming a Series or DataFrame to a new index. It essentially rearranges the existing data to match a new set of labels,

potentially introducing missing values (NaN) for labels not found in the original object or filling in existing data for labels that are present.

- **Purpose:**
  - Changing the order of rows or columns.
  - Adding or removing rows/columns.
  - Aligning data from different sources (e.g., combining time series data with different frequencies).
  - Filling in missing data using various methods.
- **Key Parameters:**
  - index (or columns for DataFrame): The new index/column labels to conform to.
  - fill_value: A scalar value to use for labels that were not present in the original object.
  - method: A filling method for missing data, such as 'ffill' (forward fill), 'bfill' (backward fill), or 'pad'.

Example:

```
import pandas as pd
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
new_index = ['c', 'a', 'd', 'b']
s_reindexed = s.reindex(new_index, fill_value=0)
print("Original Series:\n", s)
print("\nReindexed Series:\n", s_reindexed)
```

2. Dropping (.drop()):

- **Explanation:** Dropping in pandas refers to the operation of removing rows or columns from a DataFrame or Series based on their labels. It returns a *new* object with the specified labels removed, by default.
- **Purpose:** To clean or subset data by removing irrelevant rows (e.g., duplicate entries, rows with too many missing values) or columns (e.g., redundant features, columns with no predictive power).
- **Key Parameters:**
  - labels: The index labels (for rows) or column names (for columns) to drop. Can be a single label or a list of labels.
  - axis: Specifies whether to drop rows (axis=0 or 'index') or columns (axis=1 or 'columns').
  - inplace: (Optional, default False) If True, the operation modifies the DataFrame in place and returns None. If False, it returns a new DataFrame.

Example:

```
df = pd.DataFrame(np.arange(9).reshape(3, 3), columns=['A', 'B', 'C'], index=['r1', 'r2', 'r3'])
df_dropped_row = df.drop('r2', axis=0)
df_dropped_col = df.drop('B', axis=1)
print("Original DataFrame:\n", df)
print("\nDataFrame after dropping row 'r2':\n", df_dropped_row)
print("\nDataFrame after dropping column 'B':\n", df_dropped_col)
```

	3. Alignment: • **Explanation:** Alignment is the automatic process by which pandas ensures that data is correctly matched when performing operations (like arithmetic, comparisons, or merging) between two Series or DataFrames. Instead of relying on positional matching (like NumPy arrays), pandas uses the **labels** of the indexes and columns to align the data. • **Purpose:** It ensures that operations are performed on corresponding data points, even if the objects have different orders of labels or different sets of labels. This prevents errors that might arise from misaligned data and makes data integration much more robust. • **How it Works:** When an operation involves two objects, pandas effectively computes the union of their indexes (both row and column). For any label present in one object but not the other, the non-existent value is treated as NaN (unless a fill_value is specified in the operation's method, as discussed in Question 2). **Example**	
4.	**Describe DataFrames in pandas and demonstrate the following operations:** • **Creating a DataFrame** • **Accessing elements** • **Assigning values to elements** • **Checking membership of a variable** • **Removing a column** • **Filtering data** • **Constructing a DataFrame from nested dictionaries** • **Transposing a DataFrame**	**8 Marks**

	Scheme:
	• **DataFrame Description (1 mark):** Brief definition.
	• **Each operation (0.875 marks):** Code and simple output.
	Solution:
	A DataFrame in pandas is a two-dimensional labeled data structure with columns of potentially different types, analogous to a spreadsheet or a SQL table. It's the most widely used pandas object for tabular data.
	import pandas as pd
	import numpy as np
	
	# --- 1. Creating a DataFrame ---
	# From a dictionary of lists/Series
	data = {
	'Name': ['Alice', 'Bob', 'Charlie', 'David'],
	'Age': [25, 30, 35, 28],
	'City': ['NY', 'LA', 'Chicago', 'NY']
	}
	df = pd.DataFrame(data, index=['a', 'b', 'c', 'd'])
	print("--- 1. Created DataFrame ---")
	print(df)	

```
# --- 2. Accessing elements ---
# Accessing a single column
print("\n--- 2. Accessing elements (Column 'Name') ---")
print(df['Name'])

# Accessing a row by label (using .loc)
print("\n--- 2. Accessing elements (Row 'b' using .loc) ---")
print(df.loc['b'])

# Accessing an element by row and column label
print("\n--- 2. Accessing elements (Element at 'c', 'Age' using .loc) ---")
print(df.loc['c', 'Age'])

# Accessing by integer position (using .iloc)
print("\n--- 2. Accessing elements (Element at row 1, col 0 using .iloc) ---")
print(df.iloc[1, 0])

# --- 3. Assigning values to elements ---
# Assigning a value to a single element
df.loc['a', 'Age'] = 26
print("\n--- 3. Assigned value to 'a', 'Age' ---")
print(df)

# Assigning a new column
df['Salary'] = [50000, 60000, 70000, 55000]
print("\n--- 3. Assigned a new column 'Salary' ---")
print(df)

# Assigning values to an existing column using a Series (index-aligned)
new_salaries = pd.Series([62000, 72000], index=['b', 'c'])
df['Salary'] = new_salaries
print("\n--- 3. Assigned values to 'Salary' using a Series ---")
print(df)


# --- 4. Checking membership of a variable ---
# Check if a column name exists
print("\n--- 4. Checking membership ---")
print("'Age' in df.columns:", 'Age' in df.columns)
print("'Gender' in df.columns:", 'Gender' in df.columns)

# Check if an index label exists
print("'b' in df.index:", 'b' in df.index)
print("'e' in df.index:", 'e' in df.index)
```

```
# --- 5. Removing a column ---
# Using the drop method (returns a new DataFrame)
df_no_city = df.drop('City', axis=1)
print("\n--- 5. Removed 'City' column (new DataFrame) ---")
print(df_no_city)

# Alternatively, using 'del' (modifies in place)
# del df['City']
# print("\n--- 5. Removed 'City' column (using del, in place) ---")
# print(df)


# --- 6. Filtering data ---
# Filter rows where Age is greater than 30
df_filtered = df[df['Age'] > 30]
print("\n--- 6. Filtered data (Age > 30) ---")
print(df_filtered)

# Filter rows where City is 'NY'
df_filtered_city = df[df['City'] == 'NY']
print("\n--- 6. Filtered data (City == 'NY') ---")
print(df_filtered_city)


# --- 7. Constructing a DataFrame from nested dictionaries ---
# Outer keys become column names, inner keys become index names.
nested_dict = {
   'NY': {'temp': 25, 'humidity': 60},
   'LA': {'temp': 30, 'humidity': 40},
   'Chicago': {'temp': 20, 'humidity': 70}
}
df_nested = pd.DataFrame(nested_dict)
print("\n--- 7. DataFrame from nested dictionaries ---")
print(df_nested)

# --- 8. Transposing a DataFrame ---
df_transposed = df.T # Or df.transpose()
print("\n--- 8. Transposed DataFrame ---")
print(df_transposed)
```

5.	**Explain the concept of Index objects in pandas and discuss their significance in the context of Series and DataFrames.**	**7 Marks**
	Scheme: ● **Index Object Definition (2 marks):** Explain what it is (immutable array-like, labels for	

axis).

- **Significance for Series (2.5 marks):** Discuss label-based access, alignment, and ordering.
- **Significance for DataFrames (2.5 marks):** Discuss row and column labels, multi-axis alignment, and data selection.

Solution:

Concept of Index Objects:

In pandas, an Index object is an immutable (hashable) array-like structure that holds the axis labels (for rows and columns) for Series and DataFrame objects. It provides the metadata for the data, enabling efficient data retrieval, alignment, and manipulation.

Think of an Index as the "coordinates" or "keys" for your data points. It can contain duplicate labels, but typically, unique labels are preferred for easier data access and manipulation.

Significance in the Context of Series and DataFrames:

1. Significance for Series:

- **Label-Based Access:** The Index provides named labels (which can be strings, numbers, or dates) for each data point in a Series, allowing for intuitive and flexible data retrieval beyond simple integer positions. For example, series['label'] instead of series[0].
- **Automatic Data Alignment:** When performing operations between two Series (e.g., addition, subtraction), pandas automatically aligns the data based on their Index labels. This ensures that operations are performed on corresponding data points, even if the Series have different orders of labels or different sets of labels. Missing labels result in NaN.
- **Maintaining Order (Implicitly):** While an Index itself doesn't enforce strict ordering, it preserves the order of the labels as they appear in the Series, which is crucial for time series data or ordered categories.
- **Efficiency:** Pandas uses optimized C implementations for Index operations (like lookups, unions, intersections), making label-based operations very fast.

2. Significance for DataFrames:

- **Dual Labeling (Row and Column Indices):** DataFrames have two Index objects: one for the rows (df.index) and one for the columns (df.columns). This dual labeling makes DataFrames highly versatile for representing tabular data, allowing access by both row and column names.
- **Multi-Axis Data Alignment:** Similar to Series, DataFrames leverage both their row and column Index objects for automatic data alignment during operations between DataFrames. This is incredibly powerful as it ensures that the correct cells are operated upon, regardless of the order or presence of labels in the source DataFrames.
- **Powerful Data Selection:** The Index objects are integral to pandas' advanced indexing capabilities, such as .loc (label-based indexing) and .iloc (integer-position based indexing), enabling precise selection of rows, columns, or subsets of data based on their labels.
- **Data Integrity:** By providing labels, the Index helps maintain the integrity and meaning of the data when operations like sorting, reordering, or merging are performed. The labels stay associated with their data.
- **Hierarchical Indexing (MultiIndex):** Pandas extends the Index concept to MultiIndex, allowing for hierarchical or multi-level indexing. This is crucial for handling complex, multi-dimensional datasets within a 2D DataFrame structure, enabling advanced data

	pivoting and aggregation. In essence, Index objects are the "smart labels" that underpin pandas' ability to provide intuitive, flexible, and robust data manipulation capabilities.	
6.	**Describe the operations between data structures in pandas, illustrating the following:** ● **Arithmetic methods** ● **Operations involving DataFrames and Series** ● **Handling non-matching indexes.**	**6 Marks**
	Scheme: ● **Arithmetic Methods (2 marks):** Briefly explain the various arithmetic methods (e.g., add, sub, operators). ● **Operations involving DataFrames and Series (2 marks):** Explain broadcasting concept. ● **Handling non-matching indexes (2 marks):** Discuss NaN and fill_value. (Overlap with Q2, but specifically for operations between different structures). **Solution:** Pandas data structures (Series and DataFrames) are designed for highly efficient and flexible operations, especially arithmetic ones. The core principle governing these operations is **data alignment by labels** and **broadcasting**. **1. Arithmetic Methods:** Pandas provides both operator overloading (e.g., +, -, *, /) and explicit methods (.add(), .sub(), .mul(), .div()) for arithmetic operations. The methods offer more control, particularly the fill_value parameter. ● **Operator Overloading:** When you use standard Python arithmetic operators directly on Series or DataFrames, pandas performs the operation element-wise after aligning the data by their indexes (and columns for DataFrames). ● **Explicit Methods (.add(), .sub(), etc.):** These methods offer the fill_value argument to handle non-matching indices gracefully, replacing NaN with a specified value before performing the arithmetic. **2. Operations Involving DataFrames and Series:** When performing arithmetic operations between a DataFrame and a Series, pandas utilizes **broadcasting**. ● **Row-wise Broadcasting (Default):** By default, if a Series is involved in an operation with a DataFrame, the Series is broadcasted **row-wise** across the DataFrame's columns. This means the Series' index labels are matched against the DataFrame's column names. If a column in the DataFrame exists, the corresponding Series value is applied to that entire column. ● **Column-wise Broadcasting (axis parameter):** You can explicitly broadcast a Series along a DataFrame's rows by using the axis='index' or axis=0 parameter in the arithmetic method. In this case, the Series' index labels are matched against the DataFrame's row indices. **3. Handling Non-Matching Indexes:** As discussed in Question 2, when indexes (and columns for DataFrames) do not match during operations:	

	• **NaN Introduction:** Pandas forms the **union** of the labels from both data structures. For any label present in one structure but not the other, the corresponding position in the result will be filled with NaN. This behavior ensures that no data is silently lost and highlights where data was missing for the operation. **fill_value:** As mentioned, arithmetic methods allow you to specify a fill_value that will be used in place of NaN for non-matching entries. This is crucial for avoiding NaN propagation and for defining a specific behavior for missing data during operations.	
7.	**Discuss the concept of function application and mapping in pandas, illustrating the following points:** • **Element-wise operations using universal functions (ufunc)** • **Applying functions by row or column using the apply() method** • **Utilizing built-in statistical functions and generating summary statistics with describe().**	**8 Marks**

	Scheme:
	• **Ufuncs (2.5 marks):** Explanation, direct application, speed.
	• **apply() (2.5 marks):** Explanation (axis), custom functions.
	• **Built-in Stats & describe() (3 marks):** Explanation of common functions, and describe() for summary.
	Solution:
	Pandas provides powerful tools for applying functions to data, ranging from highly optimized element-wise operations to flexible row/column-wise transformations and convenient summary statistics.
	1. Element-wise Operations using Universal Functions (ufuncs):
	• **Concept:** Universal functions (ufuncs) are NumPy functions that operate element-wise on arrays. Pandas Series and DataFrames are built on NumPy arrays, so they seamlessly integrate with ufuncs. When a ufunc is applied to a Series or DataFrame, it's executed extremely efficiently, often in optimized C code, making it much faster than explicit Python loops.
	• **How it Works:** The ufunc is applied independently to each value in the Series or DataFrame, and the result is a new Series or DataFrame with the same shape, containing the results of the element-wise operation.
	• **Examples:** np.sqrt(), np.log(), np.exp(), np.abs(), np.sin(), np.cos(), np.round(), etc.
	2. Applying Functions by Row or Column using the apply() method:
	• **Concept:** The apply() method is a flexible tool for applying an arbitrary function along an axis (row-wise or column-wise) of a DataFrame or to each element of a Series. It's more general than ufuncs and can accept custom Python functions.
	• **How it Works:**
	  ○ For a DataFrame, apply() takes a function and applies it to each column (default, axis=0) or each row (axis=1). The function receives a Series (representing a column or a row) as its input.
	  ○ For a Series, apply() applies the function to each element. (This is often less efficient than ufuncs for simple element-wise ops).
	• **Examples:** Calculating custom statistics for each column/row, formatting data, or performing complex transformations that can't be done with direct vectorized operations.	

	3. Utilizing Built-in Statistical Functions and Generating Summary Statistics with describe(): ● **Concept:** Pandas Series and DataFrames have numerous built-in methods for common statistical operations, such as sum(), mean(), median(), min(), max(), std() (standard deviation), var() (variance), count(), etc. These are highly optimized. The describe() method provides a convenient, quick summary of the central tendency, dispersion, and shape of the distribution of a Series or a DataFrame's numerical columns. ● **How it Works:** ○ Built-in functions can be called directly on a Series or DataFrame (which applies them column-wise by default for DataFrames, similar to apply(axis=0)). ○ describe() computes a set of common descriptive statistics for numerical columns (and can also be used for non-numerical data with include='all')	
8.	**Explain the I/O API tools available in the pandas library for data analysis, detailing the functions categorized as readers and writers for various file formats.**	**7 Marks**
	Scheme: ● **Explanation of I/O API importance (2 marks):** Data import/export, common formats. ● **Readers and Writers categorization (5 marks):** List major functions for CSV, Excel, SQL, JSON, HTML, Parquet/Feather, and HDF5. Briefly describe purpose. **Solution:** The I/O (Input/Output) API tools in the pandas library are crucial for data analysis because they provide robust and convenient functionalities to read data from various external file formats into pandas DataFrames (readers) and to write DataFrames back into those formats (writers). This enables seamless data import, export, sharing, and persistence, making pandas a powerful tool in data pipelines. Pandas supports a wide range of file formats, each with dedicated reader and writer functions: **Readers (Functions for Reading Data into pandas DataFrames):** These functions typically start with read_ and take a file path or URL as their primary argument. 1. **CSV (Comma Separated Values):** ○ pd.read_csv(filepath_or_buffer, ...): Reads a CSV file into a DataFrame. Highly configurable for delimiters, headers, missing values, etc. 2. **Excel:** ○ pd.read_excel(io, sheet_name=0, ...): Reads data from an Excel file (XLS, XLSX) into a DataFrame. Can read specific sheets. 3. **SQL Databases:** ○ pd.read_sql_table(table_name, con, ...): Reads a SQL table into a DataFrame. ○ pd.read_sql_query(sql, con, ...): Reads the results of a SQL query into a DataFrame. ○ pd.read_sql(sql, con, ...): A convenience wrapper for both read_sql_table and read_sql_query. Requires a SQLAlchemy connection. 4. **JSON (JavaScript Object Notation):** ○ pd.read_json(path_or_buf, ...): Reads a JSON string or file into a DataFrame. Handles various JSON structures. 5. **HTML:** ○ pd.read_html(io, ...): Reads HTML tables from a URL, file, or string into a list of	

	DataFrames. Useful for web scraping. **Writers (Functions for Writing pandas DataFrames to Files):** These are methods of the DataFrame or Series object, typically starting with to_. 1. **CSV:** ○ df.to_csv(path_or_buf, ...): Writes the DataFrame to a CSV file. Options to include/exclude index, header, specify delimiter, etc. 2. **Excel:** ○ df.to_excel(excel_writer, sheet_name='Sheet1', ...): Writes the DataFrame to an Excel file. Can write to specific sheets and append to existing files using ExcelWriter. 3. **SQL Databases:** ○ df.to_sql(name, con, ...): Writes records stored in a DataFrame to a SQL database table. 4. **JSON:** ○ df.to_json(path_or_buf, ...): Writes the DataFrame to a JSON string or file. 5. **HTML:** ○ df.to_html(buf=None, ...): Writes the DataFrame as an HTML table.	
9.	**What functions does pandas offer for reading and writing data in CSV files, and how can you handle files with or without headers?**	**7 Marks**
	Scheme: ● **Reading (3 marks):** read_csv() and its parameters (header, names). ● **Writing (2 marks):** to_csv() and its parameters (header, index). ● **Handling headers (2 marks):** Explain header=None and names for no header files. **Solution:** Pandas offers robust and highly configurable functions for reading from and writing to CSV (Comma Separated Values) files, which are one of the most common formats for tabular data. **1. Reading Data from CSV Files: pd.read_csv()** The primary function for reading CSV files is pd.read_csv(). It's incredibly versatile and can handle a wide variety of CSV formats. **Key Parameters for Header Handling:** ● **header (default is 'infer'):** ○ header='infer' (or header=0): Pandas will automatically detect if the first row of the CSV file contains column names and use it as the header. This is the default and works for most standard CSVs with headers. ○ header=None: This tells pandas that the CSV file *does not* have a header row. Pandas will then assign default integer column names (0, 1, 2, ...). ○ header=int: If you specify an integer n, pandas will use the n-th row (0-indexed) as the header. ● **names (default is None):** ○ If header=None, you can use the names parameter to provide a list of column names for the DataFrame. This is crucial for giving meaningful names to columns when the file lacks a header. ○ If header is specified (e.g., header=0), names can be used in conjunction to replace the inferred header names.	

	Example **2. Writing Data to CSV Files: DataFrame.to_csv()** The DataFrame.to_csv() method is used to write a DataFrame to a CSV file. **Key Parameters for Header and Index Handling:** ● **header (default is True):** ○ header=True: Writes the column names as the first row of the CSV file. This is the default. ○ header=False: Prevents writing the column names to the CSV file. ● **index (default is True):** ○ index=True: Writes the DataFrame's row index as the first column of the CSV file. This is the default. ○ index=False: Prevents writing the row index to the CSV file. **Example**	
10.	**Illustrate how to use regular expressions in pandas to parse TXT files that contain unpredictable separators.**	**8 Marks**
	Scheme: ● **Problem with unpredictable separators (2 marks):** Explain why standard read_csv fails. ● **Solution with sep and regex (3 marks):** Explain sep parameter accepting regex. ● **Code Example (3 marks):** Create dummy file, use read_csv with regex for sep. **Solution:** When parsing text files (TXT, logs, etc.) where the data columns are separated by inconsistent or unpredictable delimiters (e.g., varying number of spaces, tabs, or a mix of special characters), the standard pd.read_csv() with a fixed sep (like sep=',' or sep='\t') will often fail or misinterpret the columns. This is where regular expressions become invaluable. Pandas' pd.read_csv() function accepts a regular expression pattern for its sep (separator) parameter. This allows you to define a more complex pattern that matches various forms of delimiters.	
11.	**Explain the process of reading text files in parts using pandas.**	**7 Marks**
	Scheme: ● **Why read in parts (2 marks):** Memory constraints, large files. ● **Process (chunksize, iterator) (3 marks):** Explain parameters, how it works. ● **Code Example (2 marks):** Demonstrate reading in chunks and iterating. **Solution:** Reading an entire text file (like a CSV) into memory at once can be problematic, especially when dealing with very large datasets that exceed available RAM. Pandas provides a mechanism to read text files in smaller, manageable chunks, allowing you to process large files incrementally without exhausting memory. **Process of Reading Text Files in Parts using pandas:** The core of reading files in parts involves using the chunksize and iterator parameters in pd.read_csv() (or other read_ functions that support it). 1. **Specify chunksize:** When you pass an integer value to the chunksize parameter in	

	pd.read_csv(), pandas will return a TextFileReader object instead of a direct DataFrame. This TextFileReader object is an iterator. 2. **Iterate through Chunks:** You can then iterate over this TextFileReader object. In each iteration, it will yield a DataFrame containing chunksize number of rows (except for the last chunk, which might contain fewer rows if the total number of rows is not perfectly divisible by chunksize). 3. **Process Each Chunk:** Inside the loop, you can perform whatever data processing or analysis you need on that smaller DataFrame chunk. This allows you to accumulate results or process data incrementally, keeping memory usage low. **Why read in parts?** ● **Memory Constraints:** The primary reason is to handle datasets that are too large to fit into your computer's RAM as a single DataFrame. ● **Performance:** For some operations, processing data in chunks can be more efficient, especially if you're only interested in certain aggregations or transformations that don't require the entire dataset in memory simultaneously. ● **Real-time Processing:** It can be useful for scenarios where data arrives in streams, and you need to process it as it comes in.	
12.	**Illustrate the functions read_html() and to_html() in pandas for reading and writing HTML files.**	**8 Marks**
	Scheme: ● **read_html() (4 marks):** Explanation (URL/file/string, returns list of DFs), practical use (web scraping), example. ● **to_html() (4 marks):** Explanation (DataFrame method, returns string/writes file), customization, example. **Solution:** Pandas provides convenient functions to interact with HTML tables, allowing you to easily extract tabular data from web pages or generate HTML tables from DataFrames. **1. pd.read_html(io, ...) (Reading HTML Tables):** ● **Purpose:** This function reads HTML tables into a list of DataFrame objects. It's incredibly useful for web scraping tabular data. ● **Input (io parameter):** Can take a URL (web page), a file path to an HTML file, or an HTML string. ● **Output:** It returns a *list* of DataFrames because an HTML document might contain multiple <table> tags. If no tables are found, it raises a ValueError. ● **Process:** Pandas uses parsing libraries (like lxml and html5lib) to identify and extract table structures from the HTML content. **2. DataFrame.to_html(buf=None, ...) (Writing DataFrames to HTML):** ● **Purpose:** This is a method of the DataFrame (or Series) object that converts the DataFrame into an HTML table string or writes it directly to an HTML file. ● **Output (buf parameter):** ○ If buf=None (default), it returns the HTML table as a string. ○ If buf is a file-like object (e.g., an open file handle), the HTML table is written directly to that file. ● **Customization:** It offers various parameters for customization, such as index (to	

include/exclude row index), header (to include/exclude column header), classes (CSS classes), border (table border), na_rep (representation for NaN values), and more.

13. Explain the functions read_excel() and to_excel() in pandas for reading from and writing to Microsoft Excel files. **7 Marks**

Scheme:

- **read_excel() (3.5 marks):** Explanation, common parameters (sheet_name, header, index_col), example.
- **to_excel() (3.5 marks):** Explanation, common parameters (excel_writer, sheet_name, index), example.

Solution:

Pandas provides robust functionalities to interact with Microsoft Excel files (both .xls and .xlsx formats) through the read_excel() function for reading and the DataFrame.to_excel() method for writing.

1. pd.read_excel(io, sheet_name=0, ...) (Reading from Excel Files):

- **Purpose:** This function reads tabular data from one or more sheets of an Excel file into a pandas DataFrame.
- **Input (io parameter):** Takes the path to an Excel file (e.g., 'path/to/my_data.xlsx').
- **Key Parameters:**
  - sheet_name (default is 0):
    - 0 (or None): Reads the first sheet by default.
    - int: Reads the sheet at the specified 0-indexed position (e.g., sheet_name=1 for the second sheet).
    - str: Reads the sheet with the given name (e.g., sheet_name='Sheet1').
    - list of int/str: Reads multiple sheets. Returns a dictionary of DataFrames, where keys are sheet names or numbers, and values are the corresponding DataFrames.
  - header (default is 0): Specifies which row (0-indexed) to use as column names. header=None means no header.
  - names: A list of column names to use if header=None or to override existing headers.
  - index_col: Specifies which column(s) to use as the row index of the DataFrame. Can be an integer, name, or list of integers/names for a MultiIndex.
  - skiprows: Number of rows to skip at the beginning of the sheet.

2. DataFrame.to_excel(excel_writer, sheet_name='Sheet1', ...) (Writing to Excel Files):

- **Purpose:** This is a method of the DataFrame (or Series) object that writes the data to an Excel file.
- **Input (excel_writer or path):** Can take a file path (string) or an ExcelWriter object. Using ExcelWriter is recommended when writing multiple DataFrames to different sheets within the same Excel file or when appending to an existing file.
- **Key Parameters:**
  - sheet_name (default is 'Sheet1'): The name of the sheet to which the DataFrame will be written.
  - index (default is True): If True, writes the DataFrame's row index as the first

	column in Excel. ○ header (default is True): If True, writes the DataFrame's column names as the first row in Excel. ○ na_rep: String representation for NaN (missing values). ○ float_format: Format string for floating point numbers.	
14.	**Given an Excel file named sales_data.xlsx containing multiple sheets (say "Q1", "Q2", "Q3"), outline the steps to perform the following tasks:** ● **Read the Excel file into a pandas dataframe.** ● **Extract data from the sheets named "Q1" and "Q2"**	**8 Marks**
	Scheme: ● **Setup (2 marks):** Create dummy Excel file. ● **Read entire file (3 marks):** Correct use of read_excel to get dict of DFs. ● **Extract specific sheets (3 marks):** Accessing dict keys. **Solution:** To perform these tasks, we'll use pd.read_excel() and its sheet_name parameter effectively. **Step 0: Create a dummy sales_data.xlsx file** (for demonstration purposes, as it's a prerequisite). import pandas as pd import os # Create dummy sales_data.xlsx with Q1, Q2, Q3 sheets with pd.ExcelWriter('sales_data.xlsx') as writer: # Q1 Data pd.DataFrame({ 'Product': ['A', 'B', 'C'], 'Sales_Q1': [100, 150, 120], 'Region': ['East', 'West', 'North'] }).to_excel(writer, sheet_name='Q1', index=False) # Q2 Data pd.DataFrame({ 'Product': ['A', 'B', 'D'], 'Sales_Q2': [110, 160, 90], 'Region': ['East', 'South', 'West'] }).to_excel(writer, sheet_name='Q2', index=False) # Q3 Data pd.DataFrame({ 'Product': ['B', 'C', 'E'], 'Sales_Q3': [170, 130, 80], 'Region': ['North', 'East', 'South'] }).to_excel(writer, sheet_name='Q3', index=False)	

	print("Dummy 'sales_data.xlsx' created with sheets 'Q1', 'Q2', 'Q3'.")
	Step 1: Read the Excel file into a pandas DataFrame (as a dictionary of DataFrames).
	To read *all* sheets or specific sheets into a dictionary where keys are sheet names and values are DataFrames, you set sheet_name=None or pass a list of sheet names to read_excel(). For this task, we want to extract "Q1" and "Q2", so providing a list is most direct.
	# Read the Excel file, specifically extracting 'Q1' and 'Q2'
	# This will return an OrderedDict where keys are sheet names and values are DataFrames.
	sales_data_dfs = pd.read_excel('sales_data.xlsx', sheet_name=['Q1', 'Q2'])
	
	print("\n--- Step 1: Excel file read into a dictionary of DataFrames ---")
	print("Type of sales_data_dfs:", type(sales_data_dfs))
	print("Sheets read (keys of the dictionary):", sales_data_dfs.keys())
	Step 2: Extract data from the sheets named "Q1" and "Q2".
	Since sales_data_dfs is now a dictionary-like object where sheet names are keys and DataFrames are values, you can simply access the DataFrames using standard dictionary key access.
	# Extract DataFrame for 'Q1' sheet
	df_q1 = sales_data_dfs['Q1']
	print("\n--- Step 2: Data from 'Q1' sheet ---")
	print(df_q1)
	
	# Extract DataFrame for 'Q2' sheet
	df_q2 = sales_data_dfs['Q2']
	print("\n--- Step 2: Data from 'Q2' sheet ---")
	print(df_q2)
	
	# You now have df_q1 and df_q2 as separate pandas DataFrames, ready for analysis.
	
	# Clean up dummy file (optional)
	# os.remove('sales_data.xlsx')	

	Module 5	
1.	**Summarize the key features of the matplotlib library that make it suitable for data visualization in scientific and engineering applications.**	**5 Marks**
	Scheme: 5 x 1 = 5 Marks **Solution:** ● Extreme simplicity in its use ● Gradual development and interactive data visualization ● Expressions and text in LaTex ● Greater control over graphic elements ● Export to many formats, such as PNG, PDF, SVG, and EPS	
2.	**Explain the architecture of the Matplotlib library and how its components interact to**	**10 Marks**

create visualizations.

Scheme:

- **Three Layers (1 mark each for name + 1 mark each for explanation = 6 marks):** Backend Layer, Artist Layer, Scripting Layer (Pyplot).
- **Interaction (4 marks):** Explain the flow from Pyplot down to the backend, how Artist objects handle rendering.

Solution:

Matplotlib's architecture can be understood in three main conceptual layers, each building upon the one below it. These layers interact to provide both high-level convenience and low-level control for creating visualizations.

1. **The Backend Layer (pyplot.backend_bases):**
    - **Purpose:** This is the foundational layer responsible for rendering. It handles the actual drawing onto a canvas (e.g., screen or file).
    - **Components:**
        - **FigureCanvas:** Defines the interface between the drawing area and the rendering backend. It handles drawing events and holds the figure.
        - **Renderer:** Is the object that actually draws on the FigureCanvas. It knows how to draw primitives (lines, text, polygons) in specific coordinate systems.
        - **Event:** Handles user input like key presses or mouse clicks, especially important for interactive plots.
    - **Interaction:** Different backends exist for different output types (e.g., Agg for PNG, PDF for PDF, TkAgg for Tkinter GUI, WebAgg for web). This layer provides the raw drawing capabilities.
2. **The Artist Layer (matplotlib.artist):**
    - **Purpose:** This layer sits above the backend and is where the "heavy lifting" of creating and arranging the visual elements of a plot happens. Everything you see on a Matplotlib figure is an Artist.
    - **Components:**
        - **Figure Artist (Figure):** The top-level container for all plot elements. It holds one or more Axes objects and contains the overall canvas.
        - **Axes Artist (Axes):** This is the most crucial Artist. It represents the actual plotting region, containing data limits, ticks, labels, and all the plot elements (lines, points, bars, patches, text). A Figure can have multiple Axes.
        - **Primitive Artists:** These are the basic drawing elements like Line2D, Text, Rectangle, Collection (for scatter plots, bar patches).
        - **Composite Artists:** These are containers for other Artists, like Figure and Axes.
    - **Interaction:** Artists know how to draw themselves (draw() method) using the Renderer from the backend layer. They manage their own transformations (data coordinates to display coordinates). When you modify a plot property (e.g., set an x-label), you are interacting with an Artist object.
3. **The Scripting Layer (matplotlib.pyplot):**
    - **Purpose:** This is the highest-level layer, designed for quick and easy plotting,

	mimicking MATLAB's plotting interface. It provides a state-based interface that automatically creates Figures and Axes for you. ○ **Components:** Functions like plt.plot(), plt.scatter(), plt.xlabel(), plt.title(), plt.show(), plt.savefig(). ○ **Interaction:** Pyplot functions implicitly create and manage Figure and Axes objects (from the Artist layer) for you. When you call plt.plot(), it gets the current Axes and adds a Line2D Artist to it. When you call plt.show(), it triggers the Figure Artist to draw() itself using the selected backend. This layer hides the complexity of the Artist and Backend layers, making it user-friendly for common plotting tasks.	
3.	**How do you create a simple line chart using Pyplot? Describe the essential functions and parameters involved.**	**10 Marks**
	Scheme: ● **Basic Setup (2 marks):** Import, data. ● **Essential Function (plt.plot()) (4 marks):** Explanation of core function, x, y parameters. ● **Customization Functions (3 marks):** plt.xlabel(), plt.ylabel(), plt.title(), plt.show(). ● **Example (1 mark):** Working code. **Solution:** Creating a simple line chart using Matplotlib's pyplot module is straightforward. It involves defining your data, using plt.plot() to draw the line, and then adding labels, a title, and displaying the plot. **Essential Functions and Parameters Involved:** 1. **import matplotlib.pyplot as plt**: This is the standard way to import the pyplot module, aliasing it as plt for convenience. 2. **plt.plot(x, y, *args, **kwargs)**: This is the core function for creating line plots (and scatter plots if markers are used without lines). ○ **x**: A sequence (list, array) of x-coordinates for the data points. ○ **y**: A sequence (list, array) of y-coordinates for the data points. ○ ***args (optional):** Allows for multiple (x, y) pairs in a single call, or a format string (e.g., 'ro-' for red circles connected by a solid line). ○ ****kwargs (optional):** Keyword arguments for customizing the line's appearance, such as: ■ color or c: Color of the line (e.g., 'red', 'blue', '#FF00FF'). ■ linestyle or ls: Style of the line (e.g., '-' solid, '--' dashed, '-.' dash-dot, ':' dotted). ■ linewidth or lw: Thickness of the line. ■ marker: Style of markers for data points (e.g., 'o' circle, '^' triangle, 's' square). ■ markersize or ms: Size of the markers. ■ label: A string label for the line, used in the legend. 3. **plt.xlabel(label, **kwargs)**: Sets the label for the x-axis. 4. **plt.ylabel(label, **kwargs)**: Sets the label for the y-axis. 5. **plt.title(title, **kwargs)**: Sets the title of the chart.	

	6. **plt.legend()**: Displays the legend on the plot. This function should be called *after* plt.plot() calls that have a label parameter. 7. **plt.grid(True, **kwargs)**: Adds a grid to the plot. True displays the grid, and **kwargs allows customization. 8. **plt.show()**: Displays the created plot. This function is essential for making the plot appear when running scripts.	
4.	**Explain how kwargs can be used to modify Pyplot plots with examples.**	**8 Marks**
	Scheme: ● **Explanation of kwargs in Python/Pyplot (3 marks):** Arbitrary keyword arguments, flexibility. ● **How kwargs modify plots (2 marks):** Passed to plotting functions, override defaults. ● **Examples (3 marks):** Demonstrate with plt.plot(), plt.xlabel() etc. **Solution:** In Python, **kwargs (keyword arguments) is a powerful construct that allows a function to accept an arbitrary number of keyword arguments. In Matplotlib's pyplot module, **kwargs are extensively used in many plotting functions to provide a flexible and concise way to modify the appearance and behavior of various plot elements. **Explanation of kwargs in Pyplot:** When you see **kwargs in the signature of a Matplotlib function (e.g., plt.plot(*args, **kwargs), plt.xlabel(label, **kwargs)), it means that the function can accept any number of additional keyword arguments that are not explicitly defined in its signature. These keyword arguments are then passed on to underlying Artist objects (like Line2D, Text, Axes) that the function creates or modifies. **How kwargs Modify Plots:** Instead of having a separate parameter for every single possible customization (which would make function signatures incredibly long and unwieldy), Matplotlib functions use **kwargs to allow users to specify properties of the plot elements. These keyword arguments directly correspond to properties of the Artists. For example, when you call plt.plot(x, y, color='red', linestyle='--'): ● color='red' is a keyword argument. plt.plot() recognizes it as a property for the Line2D Artist it creates and sets the line's color to red. ● linestyle='--' is another keyword argument that sets the line's style. This mechanism provides immense flexibility, allowing you to control almost every visual aspect of your plot elements without needing to learn separate functions for each customization. If a keyword argument is recognized by the underlying Artist, it will be applied; otherwise, it will typically be ignored or raise an error if not relevant. Examples of kwargs in Pyplot: Demonstrate parameters like color, linewidth, linestyle, fontsize, fontweight, pad, alpha, loc, frameon, and shadow are all kwargs. They offer extensive control without cluttering the function signatures.	
5.	**Illustrate how to customize a bar chart in Matplotlib by adding titles, axis labels, legends, and text annotations, and explain how to include grid lines for improved readability.**	**10 Marks**

	Scheme: • **Basic Bar Chart (2 marks):** Setup, plt.bar(). • **Title, Axis Labels, Legend (4 marks):** plt.title(), plt.xlabel(), plt.ylabel(), plt.legend(). • **Text Annotations (2 marks):** plt.text() or ax.text(). • **Grid Lines (2 marks):** plt.grid(), explanation of readability. **Solution:** Customizing a bar chart in Matplotlib involves using several pyplot functions to add descriptive elements and enhance its visual appeal and readability. Grid lines serve as visual aids on a chart. When applied, they extend from the major tick marks on the axes across the plotting area. • **Quantitative Reading:** For bar charts, horizontal grid lines are particularly useful as they allow the viewer to more easily gauge the exact height of each bar against the y-axis scale. Without grid lines, it can be challenging to precisely estimate the value represented by a bar, especially when values are close or the scale is large. • **Comparison:** Grid lines facilitate quicker comparisons between different bars by providing consistent reference points across the chart. • **Clarity:** While too many grid lines can clutter a plot, a well-placed and subtly styled grid significantly enhances the clarity and accuracy of data interpretation, making the chart more informative and user-friendly. In the example, axis='y' ensures only horizontal lines are drawn, and linestyle='--', alpha=0.6, color='gray' make them subtle rather than overpowering.	
6.	**Explain the various methods to save charts in Matplotlib, with examples.**	**8 Marks**
	Scheme: • **plt.savefig() (4 marks):** Explanation of function, file format inference, common parameters (dpi, bbox_inches). • **Figure.savefig() (2 marks):** Explanation of object-oriented approach. • **Saving to Bytes (2 marks):** Explanation (io.BytesIO). **Solution:** Matplotlib provides flexible methods to save the generated charts to various file formats, allowing for publication, sharing, or later use. The primary function for saving charts is plt.savefig() in the pyplot interface, or the Figure.savefig() method in the object-oriented interface. **1. plt.savefig(fname, *args, **kwargs) (Pyplot Interface):** This is the most commonly used method for saving plots. When you call plt.savefig(), it saves the currently active Matplotlib figure. • **fname**: The file path (string) where the plot should be saved. The file extension in fname typically determines the output format (e.g., 'my_plot.png', 'my_plot.pdf', 'my_plot.svg'). Matplotlib supports a wide range of formats like PNG, JPEG, PDF, SVG, EPS, TIFF, etc. • **Key kwargs:** ○ **dpi (dots per inch):** Controls the resolution of raster (pixel-based) output formats like PNG, JPEG. Higher DPI means higher resolution and larger file size. Default is often figure.dpi or 100. ○ **bbox_inches ('tight', None, or Bbox):** ■ 'tight' (recommended): Tries to make the bounding box of the figure	

	"tight" around the plot, automatically removing extra white space around the edges. This is highly useful for clean figures. ■ None: Uses the figure's bbox. ○ **pad_inches (float):** Amount of padding (in inches) around the bbox_inches area. Defaults to 0.1. ○ **transparent (bool):** If True, makes the background of the figure transparent. Useful for overlays. ○ **facecolor and edgecolor:** Sets the background color of the figure. ○ **format (string):** Explicitly sets the file format, overriding the inference from fname. **2. Figure.savefig(fname, *args, **kwargs) (Object-Oriented Interface):** When working with the object-oriented API (where you explicitly create Figure and Axes objects), you can call the savefig() method directly on a Figure object. This is useful when you have multiple figures open or want to be explicit about which figure you are saving. The parameters are the same as plt.savefig(). **3. Saving to a Bytes Object (In-memory Saving):** Sometimes you need to save a plot to a byte stream (e.g., to send it over a network, embed it in a web application without saving a temporary file, or store it in a database). This can be achieved using io.BytesIO.	
7.	**Explain line charts in Matplotlib, and how do you create one using a mathematical function? Discuss how to customize line styles and axes.**	**8 Marks**
	Scheme: ● **Explanation of Line Charts (2 marks):** What they represent, when to use. ● **Creating with Mathematical Function (2 marks):** Generate data, plt.plot(). ● **Customizing Line Styles (2 marks):** linestyle, linewidth, color, marker. ● **Customizing Axes (2 marks):** plt.xlim(), plt.ylim(), plt.xticks(), plt.yticks(), plt.tick_params(). **Solution:** **1. Explanation of Line Charts in Matplotlib:** A line chart (or line graph) is a type of chart that displays information as a series of data points called "markers" connected by straight line segments. It is primarily used to visualize **trends or changes over a continuous interval or time period**. Line charts are excellent for showing: ● **Trends over time:** How a variable changes over days, months, or years. ● **Relationship between two continuous variables:** How one variable responds to changes in another. ● **Comparison of multiple series:** Tracking the performance of several entities on the same scale. **2. Creating a Line Chart using a Mathematical Function:** To create a line chart from a mathematical function, you typically generate a range of input values (x-values) and then compute the corresponding output values (y-values) using the function. NumPy is commonly used to generate these numerical arrays efficiently. **3. Customizing Line Styles and Axes:** Matplotlib offers extensive parameters to customize the appearance of lines and axes for clarity	

	and visual appeal. **Customizing Line Styles:** These parameters are passed as keyword arguments to the plt.plot() function: ● **color or c**: Sets the color of the line (e.g., 'red', '#FF5733', 'blue'). ● **linestyle or ls**: Defines the pattern of the line. Common options: ○ - or 'solid': Solid line (default) ○ -- or 'dashed': Dashed line ○ -. or 'dashdot': Dash-dot line ○ : or 'dotted': Dotted line ● **linewidth or lw**: Sets the thickness of the line (a float value). ● **marker**: Specifies the style of markers at data points (e.g., 'o' circle, '^' triangle-up, 's' square, '+' plus, 'x' x). ● **markersize or ms**: Size of the markers. ● **markeredgecolor (mec)**, **markerfacecolor (mfc)**: Color of marker edge and face. ● **alpha**: Transparency of the line (0.0 for fully transparent, 1.0 for fully opaque). **Customizing Axes:** These functions are called on the pyplot module (plt) or directly on the Axes object in the object-oriented approach. ● **plt.xlim(xmin, xmax) / ax.set_xlim(xmin, xmax)**: Sets the limits of the x-axis. ● **plt.ylim(ymin, ymax) / ax.set_ylim(ymin, ymax)**: Sets the limits of the y-axis. ● **plt.xticks(ticks, labels) / ax.set_xticks(ticks) / ax.set_xticklabels(labels)**: Sets the positions and labels of x-axis ticks. Similar for yticks. ● **plt.tick_params(axis, which, direction, length, width, colors, pad, labelsize, labelcolor, bottom, top, left, right, labelbottom, labeltop, labelleft, labelright)**: Fine-grained control over tick marks and labels. ● **plt.xscale('linear'|'log'|'symlog'|'logit') / ax.set_xscale()**: Sets the scaling of the axis (e.g., logarithmic scale).	
8.	**Discuss histograms in Matplotlib.**	**8 Marks**
	Scheme: ● **Definition/Purpose (2 marks):** What a histogram is, what it shows (distribution). ● **How it works (bins) (2 marks):** Binning data. ● **plt.hist() function (2 marks):** Key parameters (bins, range, density, color, edgecolor). ● **Interpretation (2 marks):** How to read it, skewness, outliers. **Solution:** **1. Definition and Purpose:** A **histogram** is a graphical representation of the distribution of numerical data. It is an estimate of the probability distribution of a continuous variable and is a fundamental tool in exploratory data analysis (EDA). ● **Purpose:** Histograms show where the values are concentrated, where they are sparse, and what the overall shape of the distribution looks like. They are excellent for identifying: ○ The central tendency (mean, median). ○ The spread or variability of data. ○ The presence of outliers.	

- The symmetry or skewness of the distribution.
- Modality (number of peaks).

2. How it Works (Binning):

To create a histogram, the entire range of values in the dataset is divided into a series of intervals called **bins**. For each bin, the number of data points that fall into that interval (or the frequency) is counted. These counts are then represented by the height of bars in the plot.

- **Bins:** The choice of the number and width of bins is crucial. Too few bins can obscure the shape of the distribution, while too many bins can make the distribution look noisy and erratic. Matplotlib can automatically determine bin sizes, or you can specify them manually.

3. plt.hist() Function:

The matplotlib.pyplot.hist() function is used to create histograms.

Key Parameters:

- **x**: The input data, which can be a single array or a sequence of arrays (for multiple histograms).
- **bins (default 10 or 'auto'):**
  - int: The number of equal-width bins.
  - sequence: Defines the bin edges (e.g., [0, 10, 20, 30] creates bins [0,10), [10,20), [20,30)).
  - 'auto': Let Matplotlib choose the optimal bin width.
  - 'fd', 'scott', 'rice', 'sturges', 'sqrt': Algorithms for calculating optimal bin counts.
- **range (default None):** The lower and upper range of the bins. If not provided, it's inferred from x.
- **density (default False):**
  - If True, the heights of the bars are normalized so that their area sums to 1. This displays a probability density function (PDF).
  - If False (default), the heights represent counts/frequencies.
- **color:** The color of the histogram bars.
- **edgecolor:** The color of the bin edges.
- **histtype (default 'bar'):** Type of histogram to draw: 'bar', 'barstacked', 'step', 'stepfilled'.
- **alpha:** Transparency of the bars.
- **label:** Label for the histogram, used in the legend.

4. Interpretation:

When interpreting a histogram, look for:

- **Shape:** Is it symmetric (like a bell curve), skewed left (tail to the left) or skewed right (tail to the right)?
- **Center:** Where is the peak (mode) of the distribution?
- **Spread:** How wide is the distribution? Are the values clustered closely or spread out?
- **Outliers:** Are there any bars far away from the main body of the data?
- **Modality:** Does it have one peak (unimodal), two peaks (bimodal), or more (multimodal)?

For the example above, the histogram would likely show a roughly bell-shaped (normal) distribution centered around 30, with a certain spread, demonstrating how ages are distributed among the 1000 data points.

9. **Write a Python program to plot this data as a line graph. Label the x-axis as "Month" and the y-axis as "Average Temperature (°C)," and give the chart a title, "Average Monthly Temperature. he data represents the average monthly temperature (in degrees Celsius) in a city throughout the year. (8 Marks)** — **8 Marks**

Month	Average Temperature (in degrees Celsius)
January	5
February	7
March	10
April	15
May	20
June	25
July	30
August	28
September	22
October	15
November	10
December	6

Scheme:

- **Data Definition (2 marks):** Correctly define month and temperature lists/arrays.
- **Plotting (plt.plot()) (2 marks):** Correct function call.
- **Labels and Title (2 marks):** xlabel, ylabel, title.
- **Customization/Readability (1 mark):** Grid, markers.
- **plt.show() (1 mark):** Display plot.

Solution:

```
import matplotlib.pyplot as plt

# Data provided
months = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
]
average_temperatures = [5, 7, 10, 15, 20, 25, 30, 28, 22, 15, 10, 6]
```

```
# Create the line graph
plt.figure(figsize=(12, 6)) # Set figure size for better readability of month labels

# Plot the data
plt.plot(months, average_temperatures,
        marker='o',         # Add circular markers for each data point
        linestyle='-',      # Solid line
        color='skyblue',    # Line color
        linewidth=2,        # Line thickness
        label='Avg. Temperature') # Label for legend

# Label the x-axis
plt.xlabel("Month", fontsize=12, fontweight='bold', color='darkblue')

# Label the y-axis
plt.ylabel("Average Temperature (°C)", fontsize=12, fontweight='bold', color='darkgreen')

# Give the chart a title
plt.title("Average Monthly Temperature", fontsize=16, fontweight='bold', color='navy')

# Add a legend
plt.legend(loc='best', fontsize=10)

# Add grid lines for improved readability
plt.grid(True, linestyle='--', alpha=0.7)

# Rotate x-axis labels for better visibility if they overlap
plt.xticks(rotation=45, ha='right')

# Adjust layout to prevent labels from being cut off
plt.tight_layout()

# Display the chart
plt.show()
```

10. **Write a Python program to plot this data as a histogram. Label the x-axis as "Age Range" and the y-axis as "Number of Participants," and give the chart a title, "Age Distribution of Survey Participants."**

The data represents the ages of participants in a survey. (8 Marks)

Age Range	Number of Participants
18-24	50
25-34	70
35-44	40

45-54	30
55-64	20
65 and above	10

Export to Sheets

(Marks: 8)

Scheme:

- **Data Preparation (3 marks):** Convert categorical age ranges to numerical bins and expand participant counts for hist() input. This is the trickiest part.
- **Histogram Plotting (3 marks):** plt.hist() call with appropriate bins, edgecolor, alpha.
- **Labels and Title (2 marks):** xlabel, ylabel, title, xticks for custom labels.

Solution:

```
import matplotlib.pyplot as plt
import numpy as np

# Data provided
age_ranges_labels = ["18-24", "25-34", "35-44", "45-54", "55-64", "65 and above"]
number_of_participants = [50, 70, 40, 30, 20, 10]

# To use plt.hist(), we need to define the numerical bin edges and
# then "create" a flat list of ages that fall into these bins.
# We'll use the mid-point of each range to approximate the data, repeated by count.

# Define the lower edge of each bin
# For "65 and above", we'll just pick a representative upper bound, e.g., 75
bin_edges = [18, 25, 35, 45, 55, 65, 75] # The last edge extends the last bin

# Create a list of "simulated" ages based on the counts
# This is a common way to feed aggregated data to plt.hist()
# For simplicity, we'll place the ages at the lower bound of each bin for representation
# More accurately, you might distribute them randomly within the bin or use bin midpoints.
simulated_ages = []
for i, count in enumerate(number_of_participants):
    # For each participant, add an age that falls into their respective bin
    # We'll use the start of the bin range as the age for simulation
    # For "65 and above", we'll assign 65.
    if i < len(bin_edges) - 1:
        age_in_bin = bin_edges[i] # Use the lower bound of the bin
    else: # For the last bin "65 and above"
        age_in_bin = bin_edges[i] # Use 65 for the 65+ category
    simulated_ages.extend([age_in_bin] * count)

# Convert to NumPy array for better performance with hist()
```

```
simulated_ages = np.array(simulated_ages)

# Create the histogram
plt.figure(figsize=(10, 6))

# Plot the histogram.
# `bins` will use the custom bin_edges defined earlier.
# `rwidth` can make bars slightly narrower to visually separate them if desired.
plt.hist(simulated_ages,
         bins=bin_edges,
         rwidth=0.95,        # Adjust width of bars (makes them look like separated bins)
         color='lightcoral', # Bar color
         edgecolor='black',  # Border color
         alpha=0.8)          # Transparency

# Label the x-axis as "Age Range"
plt.xlabel("Age Range", fontsize=12, fontweight='bold', color='darkblue')

# Label the y-axis as "Number of Participants"
plt.ylabel("Number of Participants", fontsize=12, fontweight='bold', color='darkgreen')

# Give the chart a title
plt.title("Age Distribution of Survey Participants", fontsize=16, fontweight='bold', color='navy')

# Customize x-axis ticks to show the given age ranges
# We need to calculate the midpoints of our bins for correct label placement
# (18+25)/2 = 21.5, (25+35)/2 = 30, etc.
# For 65+, we might just place it at 67.5 or the center of bin_edges[5] and bin_edges[6]
# Here, we'll place the labels directly at the lower edge of the bins for simplicity
# Or better, we can calculate the midpoints for exact centering.
bin_midpoints = [(bin_edges[i] + bin_edges[i+1]) / 2 for i in range(len(bin_edges) - 1)]
# For "65 and above", let's use a reasonable midpoint, e.g., (65 + 75) / 2 = 70.
# The actual bins created by plt.hist with these edges are [18,25), [25,35), ..., [65,75)
plt.xticks(bin_midpoints, age_ranges_labels, rotation=45, ha='right', fontsize=10)


# Add grid lines for better readability
plt.grid(axis='y', linestyle='--', alpha=0.6)

# Ensure layout is tight
plt.tight_layout()

# Display the chart
plt.show()
```